

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO BÀI TẬP LỚN MÔN HỌC

Đề tài: Xây dựng thuật toán giải bài toán
Class Course Teacher Assignment and TimeTabling

Mã lớp học : 168497 - Nhóm 14
Học phần : Tối ưu lập kế hoạch
Mã học phần : IT4663
Giảng viên hướng dẫn : TS Nguyễn Văn Sơn

Danh sách thành viên nhóm:

STT	Họ và tên	Mã số sinh viên
1	Bùi Quang Huy	20235103
2	Nguyễn Ngọc Lê Minh	20235162
3	Nguyễn Quang Kiên	20235128

Hà Nội, tháng 05 năm 2026

LỜI CẢM ƠN

Lời đầu tiên, cho phép nhóm chúng em xin gửi lời cảm ơn chân thành đến thầy Nguyễn Văn Sơn, người đã trực tiếp truyền đạt kiến thức và tận tình hướng dẫn nhóm trong quá trình học tập cũng như thực hiện bài tập lớn này.

Thông qua sự chỉ dẫn của thầy, chúng em không chỉ có thêm những góc nhìn mới về các phương pháp, các giải thuật đối với các bài toán tối ưu, mà còn có cơ hội quý báu được tiếp cận và làm việc với các đề tài nghiên cứu hiện đại.

Nhóm chúng em rất mong nhận được những ý kiến đóng góp quý báu của các thầy để nghiên cứu của nhóm ngày càng hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

Hà Nội, ngày 28 tháng 05 năm 2026

Nhóm sinh viên thực hiện

Mục lục

1	MỞ ĐẦU	1
1.1	Giới thiệu chung	1
1.2	Bố cục đề án	1
2	PHƯƠNG PHÁP NGHIÊN CỨU	2
2.1	Giới thiệu bài toán	2
2.2	Mô hình hóa bài toán	2
2.2.1	Tập hợp và Tham số	2
2.2.2	Biến quyết định (Decision Variables)	3
2.2.3	Hàm mục tiêu (Objective Function)	3
2.2.4	Hệ Ràng buộc (Constraints)	3
2.3	Lập trình Tuyến tính Nguyên (ILP)	4
2.4	Thuật toán tham lam (Greedy Search)	4
2.5	Thuật toán di truyền kết hợp bộ giải mã tham lam ngẫu nhiên	6
2.5.1	Biểu diễn cá thể	7
2.5.2	Tiền xử lý dữ liệu	7
2.5.3	Cấu trúc thời khóa biểu và kiểm tra ràng buộc	8
2.5.4	Khởi tạo quần thể	9
2.5.5	Giải mã chromosome bằng tham lam ngẫu nhiên	9
2.5.6	Hàm thích nghi	10
2.5.7	Các toán tử tiến hóa	11
2.5.8	Elitism và cơ chế restart	12
2.5.9	Kiểm tra tính hợp lệ của nghiệm	13
2.5.10	Sơ đồ luồng thuật toán	13
2.5.11	Thiết kế thí nghiệm	13
2.5.12	Nhận xét	15
2.6	Thuật toán Hill Climbing Local Search (HCLS)	15
2.6.1	Hàm mục tiêu (Fitness Function)	15
2.6.2	Khởi tạo Lời giải	16
2.6.3	Các Toán tử Lân cận	16
2.6.4	Vòng quay Adaptive và chọn điểm	17
2.6.5	Cơ chế dừng	18
2.7	Adaptive Large Neighbourhood Search (ALNS)	18
2.7.1	Khởi tạo nghiệm	18
2.7.2	Pha phá hủy và tái xây dựng	19
2.7.3	Cơ chế chấp nhận nghiệm	19
3	THỰC NGHIỆM	21
3.1	Dữ liệu	21
3.2	Môi trường thực nghiệm	22

4	KẾT QUẢ THỰC NGHIỆM	23
4.1	Bộ giải chính xác nguyên (ILP)	23
4.2	Thuật toán tham lam	24
4.3	Thuật toán HCLS	25
4.4	Thuật toán ALNS	28
4.5	Kết quả thực nghiệm thuật toán di truyền kết hợp bộ giải mã tham lam ngẫu nhiên	31
4.5.1	Thiết lập thí nghiệm	31
4.5.2	Kết quả tổng hợp theo cấu hình	32
4.5.3	So sánh kết quả theo từng bộ dữ liệu	34
4.5.4	Ảnh hưởng của chế độ khởi tạo	35
4.5.5	Ảnh hưởng của xác suất đột biến	36
4.5.6	So sánh toán tử lai ghép OX và PMX	37
4.5.7	Phân tích hội tụ	38
4.5.8	Thời gian chạy	41
4.5.9	Nhận xét chung	42
5	KẾT LUẬN	44
6	Phụ lục	45
6.1	Mã giả thuật toán	45
6.2	Bảng thống kê kết quả	47
	Tài liệu tham khảo	57

1 MỞ ĐẦU

1.1 Giới thiệu chung

Bài toán Class Course Teacher Assignment and TimeTabling là một bài toán quan trọng trong lớp bài toán nghiên cứu về lập kế hoạch. Bài toán là sự kết hợp của hai bài toán con, đó là:

- **Phân công giáo viên (Teacher Assignment):** Quyết định giáo viên nào sẽ phụ trách lớp học/môn học nào dựa trên năng lực chuyên môn, định mức giờ dạy.
- **Xếp thời khóa biểu (TimeTabling):** Phân bổ các tiết dạy đã được phân công vào các khung thời gian (slots) và phòng học cụ thể trong tuần.

Việc tìm ra phương pháp hiệu quả để giải quyết lớp bài toán này giúp ích rất nhiều trong thực tiễn, đặc biệt ở các trường học khi mà việc xếp lịch bằng thủ công trong khi số lượng lớp, học sinh càng ngày nhiều lên, gây ra rất nhiều sự lãng phí về thời gian, tài nguyên. Bài báo cáo này sẽ tập trung giới thiệu, phân tích và đánh giá hiệu năng các phương pháp để giải quyết bài toán trên với kích thước số lớp từ 10 đến 1000 lớp học.

1.2 Bố cục đề án

Đề án ngoài phần mở đầu, tài liệu tham khảo và kết luận, được chia làm 3 phần chính:

- **Phần 1:** Giới thiệu chung về bài toán, mục tiêu của báo cáo
- **Phần 2:** Giới thiệu chi tiết về bài toán và các phương pháp giải quyết bài toán
- **Phần 3:** Thực nghiệm và đánh giá kết quả.

2 PHƯƠNG PHÁP NGHIÊN CỨU

2.1 Giới thiệu bài toán

Bài toán Phân công Giáo viên và Xếp Thời khóa biểu (TATT) liên quan đến việc phân công một tập hợp các lớp-môn cho một tập hợp các giáo viên khả dụng, đồng thời lập lịch vào các tiết học cụ thể sao cho thỏa mãn các ràng buộc và tối ưu hóa các mục tiêu nhất định. Mỗi lớp học có một danh sách các môn học bắt buộc và mỗi giáo viên có một danh sách các môn có thể đảm nhiệm trong quỹ thời gian 5 ngày một tuần (12 tiết một ngày).

Mục tiêu là tìm ra một phương án phân công và xếp lịch khả thi đáp ứng các yêu cầu chính sau:

- **Ràng buộc không chồng lấn (Overlap constraint):** Các lớp-môn của cùng một lớp, cũng như các lớp-môn được phân công cho cùng một giáo viên, không được phép trùng lặp về mặt thời gian trên thời khóa biểu.
- **Ràng buộc năng lực (Eligibility constraint):** Giáo viên chỉ được phân công giảng dạy các môn học nằm trong danh sách chuyên môn cho phép của mình.
- **Mục tiêu tối ưu hóa (Optimization objective):** Tối đa hóa tổng số lượng lớp-môn được phân xếp thời gian và gán giáo viên thành công.

Bài toán TATT được biết đến là một bài toán NP-hard. Do đó, các phương pháp tiếp cận thực tế thường dựa trên các thuật toán heuristic, xấp xỉ, hoặc các giải thuật tối ưu hóa để thu được lời giải chất lượng tốt trong một khoảng thời gian hợp lý.

2.2 Mô hình hóa bài toán

2.2.1 Tập hợp và Tham số

- $N = \{1, 2, \dots, N\}$: Tập hợp các nhóm sinh viên (Lớp).
- C_n : Tập hợp các môn học bắt buộc của lớp n .
- T_m : Tập hợp các giáo viên có chuyên môn giảng dạy môn m .
- $d_m \in \mathbb{Z}^+$: Số kíp học liên tiếp cần thiết cho môn m .
- $\Omega = \{(n, m) \mid n \in N, m \in C_n, |T_m| > 0\}$: Tập hợp các khối Lớp-Môn hợp lệ (chỉ bao gồm các môn có ít nhất một giáo viên có thể giảng dạy).
- $U = \{1, 2, \dots, 60\}$: Tập hợp toàn bộ các kíp học trong tuần (5 ngày $\times$ 12 kíp).

2.2.2 Biến quyết định (Decision Variables)

- $S_{n,m} \in \{0, 1\}$: Biến trạng thái mục tiêu, nhận giá trị 1 nếu khối $(n, m) \in \Omega$ được xếp lịch thành công, và bằng 0 nếu ngược lại.
- $V_{n,m,t,u} \in \{0, 1\}$: Biến phân công chi tiết, nhận giá trị 1 nếu khối $(n, m) \in \Omega$ được giao cho giáo viên $t \in T_m$ giảng dạy và bắt đầu chính xác tại kíp $u \in U$. Bằng 0 nếu ngược lại.

2.2.3 Hàm mục tiêu (Objective Function)

Mục tiêu của bài toán là tối đa hóa tổng số khối Lớp-Môn hợp lệ được sắp xếp lịch thành công:

$$\max Z = \sum_{(n,m) \in \Omega} S_{n,m} \quad (1)$$

2.2.4 Hệ Ràng buộc (Constraints)

1. Ràng buộc Nhất quán và Phân công: Mỗi khối Lớp-Môn hợp lệ, nếu được quyết định xếp lịch, thì phải được phân công cho đúng một giáo viên có chuyên môn và bắt đầu tại đúng một kíp duy nhất trên dòng thời gian:

$$\sum_{t \in T_m} \sum_{u \in U} V_{n,m,t,u} = S_{n,m} \quad \forall (n, m) \in \Omega \quad (2)$$

2. Ràng buộc Chống đè lịch Lớp: Đảm bảo tại bất kỳ kíp học s nào, một lớp chỉ được phép tham gia tối đa một môn học. Một môn học bắt đầu ở kíp u sẽ chiếm dụng quỹ thời gian từ kíp u đến kíp $u + d_m - 1$:

$$\sum_{m \in C_n} \sum_{t \in T_m} \sum_{\substack{u \in U: \\ u \leq s < u + d_m}} V_{n,m,t,u} \leq 1 \quad \forall n \in N, \forall s \in U \quad (3)$$

3. Ràng buộc Chống đè lịch Giáo viên: Đảm bảo tại bất kỳ kíp học s nào, một giáo viên chỉ được phép giảng dạy tối đa một lớp:

$$\sum_{n \in N} \sum_{\substack{m \in C_n: \\ t \in T_m}} \sum_{\substack{u \in U: \\ u \leq s < u + d_m}} V_{n,m,t,u} \leq 1 \quad \forall t \in T, \forall s \in U \quad (4)$$

4. Ràng buộc Chống vắt chéo buổi học (Miền thời gian hợp lệ): Một tuần có 5 ngày, mỗi ngày gồm 2 buổi (sáng/chiều) và mỗi buổi kéo dài đúng 6 kíp. Để một môn học có độ dài d_m kíp không bị vắt chéo từ buổi này sang buổi khác, kíp bắt đầu u phải đảm bảo rằng khoảng thời gian từ u đến $u + d_m - 1$ nằm trọn trong cùng một buổi. Hệ

phương trình sau ép tổng tất cả các trạng thái phân công rơi vào kíp vi phạm bằng 0:

$$\sum_{\substack{u \in U: \\ ((u-1) \bmod 6) + d_m > 6}} V_{n,m,t,u} = 0 \quad \forall (n, m) \in \Omega, \forall t \in T_m \quad (5)$$

2.3 Lập trình Tuyến tính Nguyên (ILP)

Phương pháp Lập trình Tuyến tính Nguyên (ILP) cung cấp một giải pháp chính xác để giải quyết bài toán phân công bằng cách khai thác mô hình toán học đã được trình bày trong Mục III. Phương pháp này biểu diễn bài toán dưới dạng một hệ các ràng buộc tuyến tính kết hợp với một hàm mục tiêu nhằm tối ưu hóa các điều kiện đặt ra, đảm bảo tìm được phương án phân công tối ưu và thỏa mãn tuyệt đối mọi ràng buộc của bài toán. Do bản chất NP-hard của bài toán, tiếp cận ILP đặc biệt hiệu quả đối với các bài toán có quy mô nhỏ nhưng sẽ gặp phải thách thức lớn về khả năng mở rộng (scalability) khi quy mô dữ liệu tăng lên. Mục này sẽ trình bày chi tiết cấu hình mô hình ILP và quy trình triển khai thực nghiệm sử dụng ba bộ giải (solvers) phổ biến bao gồm: OR-Tools CP-SAT [?], OR-Tools Linear Programming (`pywraplp`) [?], và bộ giải thương mại Gurobi [?]. Trên cơ sở đó, chúng tôi tiến hành đánh giá ưu điểm cũng như hạn chế của từng bộ giải, làm tiền đề cho các phân tích so sánh hiệu năng với các phương pháp heuristic và meta-heuristic ở các mục tiếp theo.

Bảng 1: So sánh đặc điểm của các bộ giải (Solvers)

Bộ giải	Chiến lược tìm kiếm	Tốc độ
<code>pywraplp</code>	Simplex, Branch-and-Bound	Trung bình
CP-SAT	Tìm kiếm dựa trên SAT	Nhanh (Rời rạc)
Gurobi	Branch-and-Cut, Heuristics (Giải thuật toán)	Rất nhanh

Tổng quan về đặc tính của ba bộ giải được mô tả chi tiết trong Bảng 1. Trong đó, `pywraplp` thuộc thư viện OR-Tools của Google, hoạt động hiệu quả khi giải quyết các bài toán lập trình tuyến tính và lập trình nguyên hỗn hợp cơ bản, nhưng tốc độ hội tụ có xu hướng chậm lại đối với các bài toán quy mô lớn.

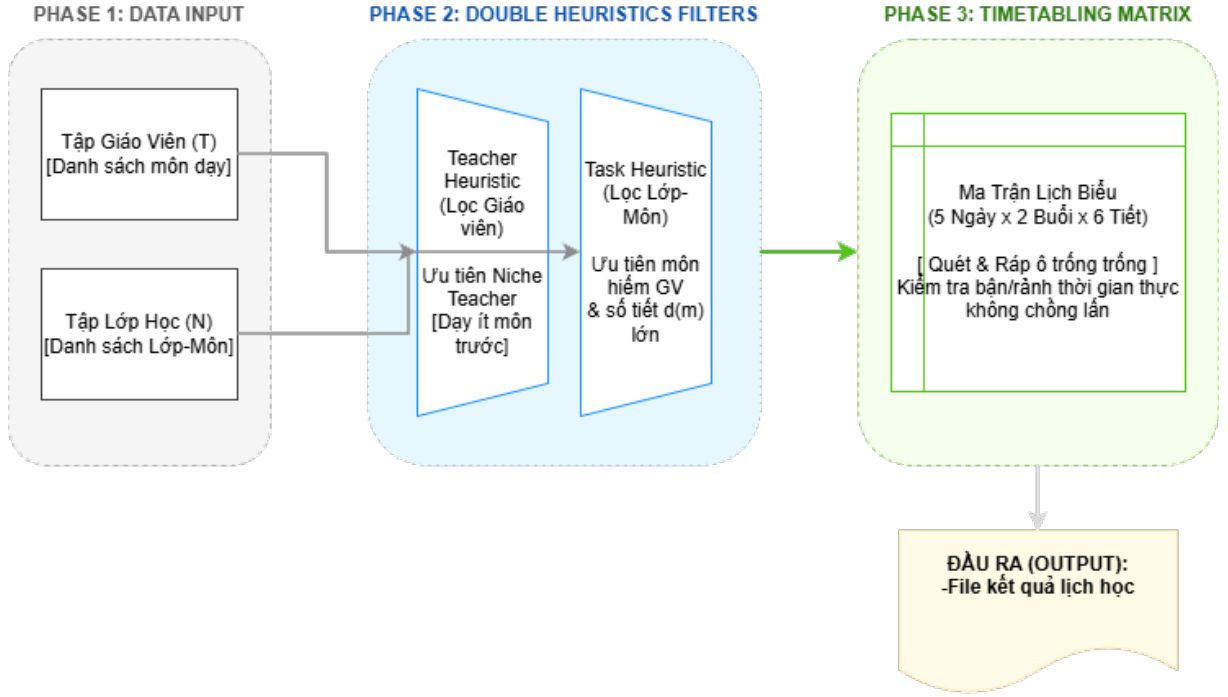
Bộ giải CP-SAT, cũng thuộc hệ sinh thái OR-Tools, được thiết kế chuyên biệt để xử lý các bài toán logic và tối ưu hóa dựa trên ràng buộc (Constraint Programming). Cuối cùng, Gurobi là một bộ giải thương mại mạnh mẽ, có khả năng xử lý các bài toán quy mô lớn và cấu trúc phức tạp một cách cực kỳ hiệu quả, bao gồm cả lập trình tuyến tính thuần túy lẫn lập trình nguyên. Tuy nhiên, Gurobi yêu cầu bản quyền thương mại để sử dụng, mặc dù có cung cấp các gói học thuật miễn phí dành cho sinh viên và các nhà nghiên cứu.

2.4 Thuật toán tham lam (Greedy Search)

Thuật toán tham lam là một phương pháp heuristic đơn giản để đưa ra lời giải tốt trong thời gian ngắn. Trong bài toán này, chiến lược tham lam là tác động đồng thời lên cả

danh sách giáo viên và danh sách nhiệm vụ cần xếp lịch. Quy trình thực hiện của thuật toán bao gồm các bước sau:

1. **Chiến lược ưu tiên Giáo viên chuyên biệt (Teacher Heuristic):** Đối với mỗi môn học, danh sách các giáo viên có khả năng giảng dạy được sắp xếp theo thứ tự tăng dần về số lượng môn học mà họ có thể đảm nhiệm. Thuật toán sẽ ưu tiên phân công những giáo viên bầm lớp có chuyên môn hẹp trước (*Niche Teachers*), nhằm mục đích "để dành" các giáo viên đa năng (*Super Teachers* - những người dạy được nhiều môn) cho các lớp học phức tạp và khó điều phối ở giai đoạn sau.
2. **Chiến lược ưu tiên Nhiệm vụ (Task Heuristic):** Toàn bộ các khối Lớp-Môn cần xếp lịch được đưa vào một danh sách hàng đợi và được định tuyến ưu tiên dựa trên hai tiêu chí:
 - *Tiêu chí 1 (Độ khan hiếm nhân lực):* Khối Lớp-Môn nào có ít giáo viên có khả năng dạy nhất sẽ được đẩy lên xếp lịch trước. Điều này giúp giải quyết triệt để các rủi ro tắc nghẽn lịch trình do thiếu hụt nhân sự chuyên môn.
 - *Tiêu chí 2 (Độ dài môn học):* Trong trường hợp có cùng số lượng giáo viên khả dụng, môn học nào có số tiết cấu hình (d_m) dài hơn sẽ được ưu tiên xếp trước. Chiến lược giúp tận dụng tối đa các khoảng trống lớn của thời khóa biểu ngay từ đầu.
3. **Cơ chế ráp lịch và Kiểm tra xung đột thời gian thực:** Sau khi danh sách nhiệm vụ và nhân sự đã được định hình theo các chiến lược ưu tiên, thuật toán tiến hành duyệt tuần tự. Với mỗi Lớp-Môn, hệ thống sẽ quét qua 10 buổi học trong tuần ($5 \text{ ngày} \times 2 \text{ buổi}$) và tìm kiếm vị trí lớp bắt đầu hợp lệ sao cho trong suốt thời lượng d_m tiết liên tiếp, cả Lớp học và Giáo viên được chọn đều đang ở trạng thái rảnh (*False*). Ngay khi tìm được vị trí đầu tiên thỏa mãn, lịch trình sẽ được chốt chặn, trạng thái bận (*True*) được cập nhật và thuật toán chuyển sang nhiệm vụ tiếp theo.



Hình 1: Sơ đồ thuật toán tham lam

- **Time:** $O(|Tasks| \times |Teachers| \times |Slots|) \approx$ mili-giây
- **Space:** $O(N \times T)$ (2 bitmap)

2.5 Thuật toán di truyền kết hợp bộ giải mã tham lam ngẫu nhiên

Trong bài toán này, mục tiêu là phân công giáo viên và kíp học cho các cặp lớp-môn sao cho thỏa mãn các ràng buộc về thời khóa biểu, đồng thời tối đa hóa số lớp-môn được xếp thành công. Bài toán có tính tổ hợp rất cao do mỗi lớp-môn cần đồng thời lựa chọn giáo viên phù hợp và kíp bắt đầu học lệ trong thời khóa biểu. Vì vậy, nhóm sử dụng một thuật toán di truyền thuần, kết hợp với bộ giải mã tham lam ngẫu nhiên hóa, để tìm kiếm các thứ tự xếp lịch tốt.

Thuật toán được triển khai là *Pure Genetic Algorithm with Randomized Greedy Decoder*. Điểm quan trọng là thuật toán không mã hóa trực tiếp toàn bộ thời khóa biểu trong chromosome. Nếu mã hóa trực tiếp giáo viên và kíp học cho từng lớp-môn, cá thể sinh ra sau lai ghép hoặc đột biến rất dễ vi phạm ràng buộc, ví dụ trùng lịch lớp, trùng lịch giáo viên hoặc môn học bị vắt qua buổi. Thay vào đó, chromosome chỉ mã hóa thứ tự ưu tiên xếp các cặp lớp-môn. Sau đó, bộ giải mã tham lam ngẫu nhiên hóa sẽ lần lượt duyệt theo thứ tự này để xây dựng một thời khóa biểu hợp lệ.

Cách tiếp cận này giúp tách bài toán thành hai tầng:

- Tầng di truyền: tìm kiếm trên không gian các hoán vị lớp-môn, tức là tìm thứ tự xếp lịch tốt.

- Tầng giải mã: chuyển mỗi hoán vị thành một thời khóa biểu cụ thể bằng cách chọn giáo viên và kíp học hợp lệ.

Nhờ đó, các toán tử di truyền như chọn lọc, lai ghép và đột biến có thể thao tác trên chromosome một cách đơn giản, trong khi tính hợp lệ của thời khóa biểu được kiểm soát ở bước giải mã.

2.5.1 Biểu diễn cá thể

Mỗi lớp-môn được biểu diễn bởi một cặp: (c, m) , trong đó c là chỉ số lớp học và m là chỉ số môn học mà lớp đó cần học. Nếu toàn bộ bài toán có K cặp lớp-môn cần xếp, một chromosome được biểu diễn là một hoán vị: $[(c_1, m_1), (c_2, m_2), \dots, (c_K, m_K)]$.

Thứ tự xuất hiện của các cặp lớp-môn trong chromosome quyết định thứ tự mà thuật toán thử xếp lịch. Các lớp-môn đứng trước trong chromosome sẽ được ưu tiên phân công giáo viên và kíp học trước. Đây là yếu tố có ảnh hưởng lớn đến chất lượng nghiệm, vì những lớp-môn khó nếu bị xếp quá muộn có thể không còn giáo viên hoặc khoảng thời gian phù hợp.

Trong chương trình, mỗi cá thể gồm hai thành phần:

- **order:** chromosome, là một hoán vị của các cặp lớp-môn;
- **schedule:** thời khóa biểu thu được sau khi giải mã chromosome.

Ví dụ, một chromosome có thể có dạng:

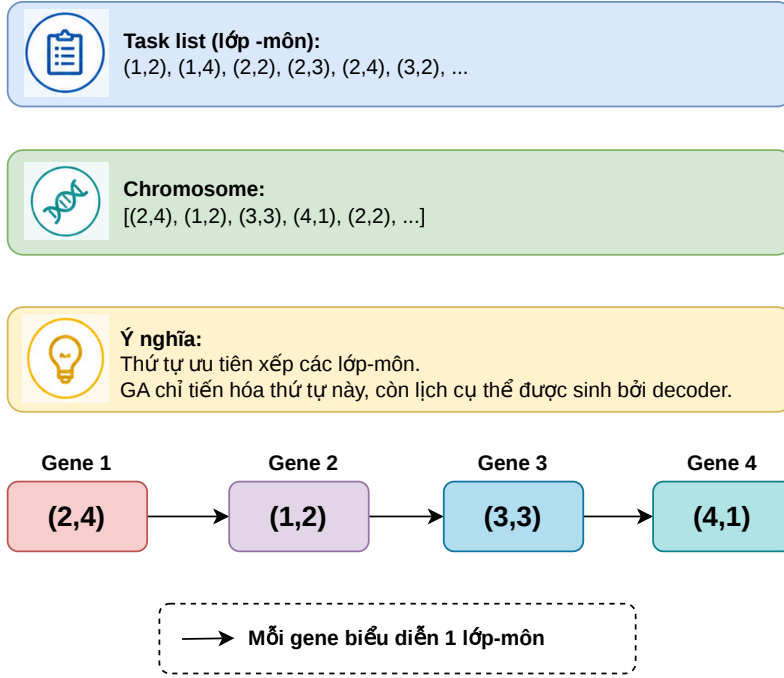
$$[(1, 2), (3, 4), (2, 1), (5, 3), \dots]$$

Điều này có nghĩa là thuật toán sẽ thử xếp môn 2 cho lớp 1 trước, sau đó đến môn 4 cho lớp 3, tiếp theo là môn 1 cho lớp 2, v.v.

2.5.2 Tiền xử lý dữ liệu

Trước khi chạy thuật toán di truyền, chương trình thực hiện một số bước tiền xử lý nhằm tăng tốc quá trình giải mã và kiểm tra ràng buộc:

- Tạo danh sách tất cả các cặp lớp-môn cần xếp lịch.
- Với mỗi môn học, xác định danh sách các giáo viên có thể giảng dạy môn đó.
- Với mỗi giáo viên, lưu các môn mà giáo viên đó có thể dạy.
- Với mỗi môn học, xác định danh sách các kíp bắt đầu hợp lệ.
- Tạo thứ tự heuristic cơ sở **base_order** để ưu tiên các lớp-môn khó.
- Tạo thứ tự giáo viên cơ sở cho từng môn học.



Hình 2: Mã hóa chromosome của thuật toán GA

Do thời khóa biểu gồm 5 ngày, mỗi ngày có 2 buổi, mỗi buổi có 6 tiết, tổng số slot là: $5 \times 2 \times 6 = 60$. Một môn học chỉ được phép bắt đầu tại các kíp sao cho toàn bộ thời lượng của môn nằm trong cùng một buổi học. Nói cách khác, môn học không được vắt qua ranh giới giữa hai buổi. Với mỗi môn có thời lượng d_m , chương trình sinh danh sách các kíp bắt đầu hợp lệ trong từng buổi, từ tiết đầu đến tiết $6 - d_m + 1$ của buổi đó.

Thứ tự heuristic cơ sở được xây dựng theo nguyên tắc ưu tiên các lớp-môn khó xếp trước. Cụ thể, các tiêu chí được dùng gồm:

- Môn có ít giáo viên dạy được được ưu tiên hơn.
- Môn có thời lượng dài hơn được ưu tiên hơn.
- Lớp có nhiều môn cần học hơn được ưu tiên hơn.

Cách sắp xếp này giúp tăng khả năng xếp được các lớp-môn khó trước khi thời khóa biểu bị chiếm bởi các môn dễ hơn.

2.5.3 Cấu trúc thời khóa biểu và kiểm tra ràng buộc

Để kiểm tra nhanh xung đột lịch, chương trình sử dụng hai cấu trúc đánh dấu:

`class_busy[c][s]`, `teacher_busy[t][s]`,

trong đó c là lớp học, t là giáo viên và s là slot thời gian.

Khi thử xếp một lớp-môn (c, m) cho giáo viên t tại kíp bắt đầu u , thuật toán kiểm tra toàn bộ d_m slot liên tiếp kể từ u . Việc xếp lịch chỉ được chấp nhận nếu:

$$\text{class_busy}[c][s] = 0 \text{ và } \text{teacher_busy}[t][s] = 0$$

với mọi slot s trong khoảng thời gian của môn học.

Nếu điều kiện trên thỏa mãn, lớp-môn được đưa vào lịch, đồng thời các slot tương ứng của lớp và giáo viên được đánh dấu là bận. Nhờ vậy, nghiệm sinh ra sau bước giải mã luôn thỏa mãn hai ràng buộc chính:

- Các lớp-môn của cùng một lớp không được chồng lấp thời khóa biểu.
- Các lớp-môn được phân công cho cùng một giáo viên không được chồng lấp thời khóa biểu.

2.5.4 Khởi tạo quần thể

Thuật toán hỗ trợ hai chế độ khởi tạo:

- **heuristic**: đưa nghiệm greedy baseline vào quần thể, sau đó sinh thêm các cá thể kiểu **noisy**, **hard_first** và **random**.
- **random**: không đưa nghiệm greedy vào quần thể, quần thể ban đầu được sinh ngẫu nhiên để quan sát rõ hơn quá trình hội tụ của thuật toán di truyền.

Trong chế độ **heuristic**, nghiệm greedy baseline được tạo bằng cách giải mã thứ tự heuristic cơ sở. Nghiệm này đóng vai trò là nghiệm nền tốt ban đầu. Sau đó, các cá thể khác được sinh ra nhằm tăng độ đa dạng của quần thể:

- **Greedy**: sử dụng đúng thứ tự heuristic cơ sở.
- **Noisy**: bắt đầu từ thứ tự heuristic nhưng thêm nhiều ngẫu nhiên vào vị trí các lớp-môn.
- **Hard-first**: ưu tiên các lớp-môn khó, nhưng vẫn có thành phần ngẫu nhiên.
- **Random**: xáo trộn hoàn toàn danh sách lớp-môn.

Việc kết hợp giữa cá thể có định hướng heuristic và cá thể ngẫu nhiên giúp thuật toán cân bằng giữa khai thác nghiệm tốt ban đầu và khám phá các vùng nghiệm mới.

2.5.5 Giải mã chromosome bằng tham lam ngẫu nhiên

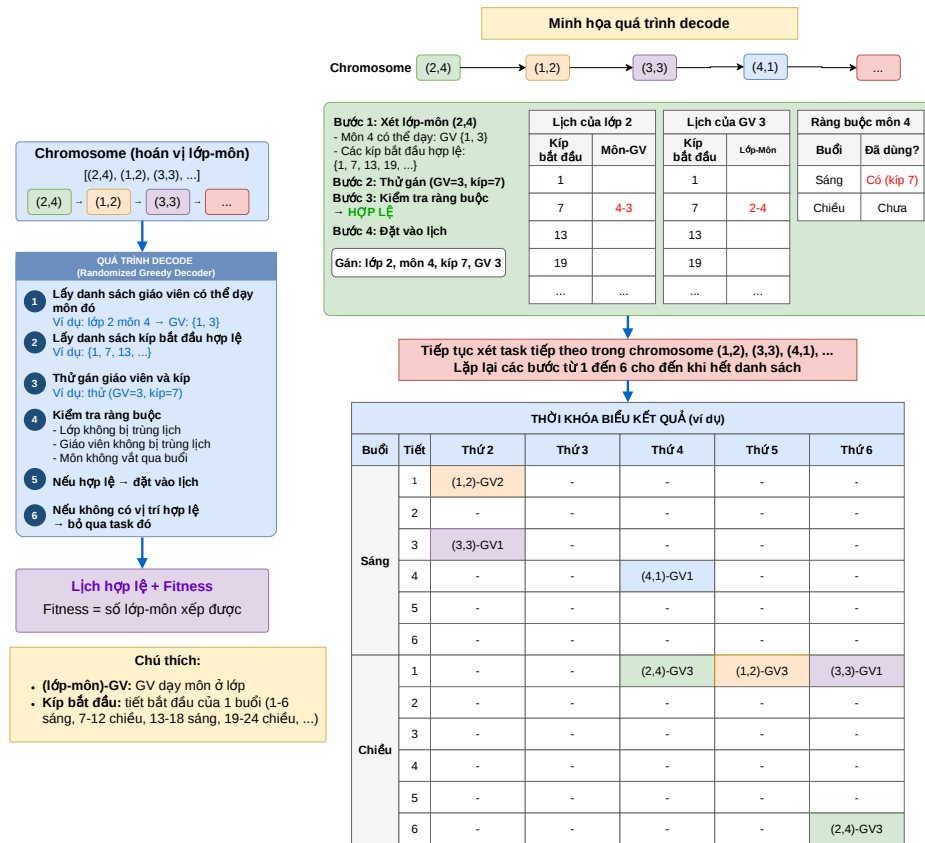
Sau khi có chromosome, thuật toán cần chuyển thứ tự lớp-môn thành một thời khóa biểu cụ thể. Quá trình này được thực hiện bằng bộ giải mã *randomized greedy*.

Với mỗi cặp lớp-môn (c, m) trong chromosome, thuật toán thực hiện các bước sau:

1. Lấy thời lượng d_m của môn học.
2. Lấy danh sách giáo viên có thể dạy môn m .

3. Lấy danh sách các kíp bắt đầu hợp lệ của môn m .
4. Sắp xếp hoặc xáo trộn danh sách giáo viên và kíp học theo một mức ngẫu nhiên `random_level`.
5. Duyệt từng giáo viên và từng kíp học.
6. Nếu lớp c và giáo viên t đều rảnh trong toàn bộ d_m slot liên tiếp thì xếp lớp-môn này vào lịch.
7. Nếu không tìm được vị trí hợp lệ thì lớp-môn đó không được xếp trong nghiệm hiện tại.

Điểm khác biệt so với greedy thông thường là quá trình giải mã có yếu tố ngẫu nhiên. Với `random_level` khác nhau, cùng một chromosome vẫn có thể tạo ra các thời khóa biểu khác nhau. Điều này giúp tăng khả năng khám phá và giảm nguy cơ thuật toán chỉ lặp lại một lịch cố định.



Hình 3: Bộ giải mã của thuật toán GA

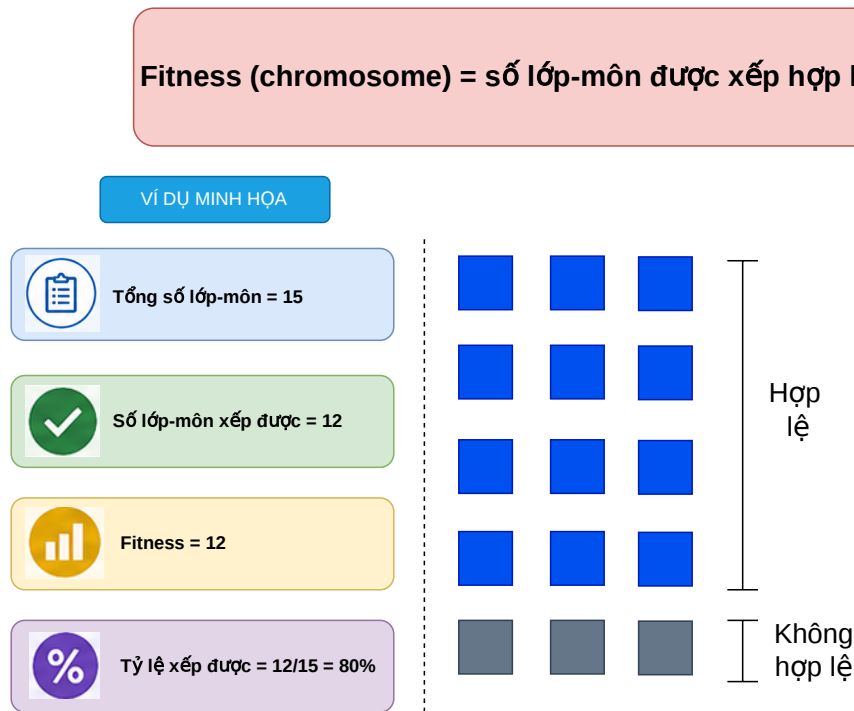
2.5.6 Hàm thích nghi

Hàm thích nghi được dùng để đánh giá chất lượng của mỗi cá thể. Vì mục tiêu của bài toán là tối đa hóa số lớp-môn được phân công giáo viên và kíp học thành công, fitness của một cá thể được định nghĩa là: $fitness = |\text{assignments}|$.

Trong đó, **assignments** là tập các lớp-môn đã được xếp lịch hợp lệ. Nếu tổng số lớp-môn của bộ dữ liệu là K , và cá thể xếp được K' lớp-môn, thì tỉ lệ xếp thành công là:

$$success_rate = \frac{K'}{K}.$$

Cá thể có fitness càng cao thì số lớp-môn được xếp lịch càng nhiều, do đó càng được ưu tiên trong quá trình chọn lọc và giữ lại qua các thế hệ.



Hình 4: Hàm Fitness của thuật toán GA

2.5.7 Các toán tử tiến hóa

Thuật toán sử dụng các toán tử di truyền phù hợp với biểu diễn chromosome dạng hoán vị.

Chọn lọc. Thuật toán sử dụng *tournament selection*. Ở mỗi lần chọn, chương trình lấy ngẫu nhiên k cá thể trong quần thể, sau đó chọn cá thể có fitness cao nhất trong nhóm đó làm cá thể cha hoặc mẹ. Trong các thí nghiệm, giá trị được sử dụng là: $k = 3$.

Cách chọn này tạo áp lực chọn lọc vừa phải: cá thể tốt có xác suất được chọn cao hơn, nhưng các cá thể khác vẫn có cơ hội tham gia sinh sản.

Lai ghép. Do chromosome là một hoán vị của các cặp lớp-môn, toán tử lai ghép phải đảm bảo cá thể con vẫn là một hoán vị hợp lệ, không bị mất hoặc lặp lớp-môn. Chương trình hỗ trợ ba kiểu lai ghép:

- **Order Crossover (OX):** chọn một đoạn liên tiếp từ cha thứ nhất, sau đó điền các vị trí còn lại theo thứ tự xuất hiện trong cha thứ hai.

- **Partially Mapped Crossover (PMX):** giữ một đoạn ánh xạ giữa hai cha, sau đó hoàn thiện các vị trí còn lại sao cho cá thể con vẫn là một hoán vị hợp lệ.
- **Position-based Crossover:** giữ ngẫu nhiên một số vị trí từ cha thứ nhất, phần còn lại được điền theo thứ tự từ cha thứ hai.

Trong các thí nghiệm chính, nhóm tập trung so sánh hai kiểu lai ghép là OX và PMX.

Đột biến. Thuật toán sử dụng các phép đột biến trên hoán vị:

- **Swap mutation:** hoán đổi vị trí của hai lớp-môn.
- **Insert mutation:** lấy một lớp-môn ra khỏi vị trí hiện tại và chèn vào vị trí khác.
- **Reverse mutation:** đảo ngược thứ tự của một đoạn trong chromosome.
- **Scramble mutation:** xáo trộn ngẫu nhiên thứ tự của một đoạn trong chromosome.

Khi sử dụng chế độ *mixed*, chương trình lựa chọn ngẫu nhiên giữa các loại đột biến trên. Cụ thể:

- 35
- 30
- 20
- 15

Các phép đột biến này giúp thay đổi thứ tự ưu tiên xếp lịch, từ đó tạo ra các khả năng phân công giáo viên và lớp học khác nhau sau bước giải mã.

2.5.8 Elitism và cơ chế restart

Sau mỗi thế hệ, quần thể được sắp xếp theo fitness. Một số cá thể tốt nhất được giữ lại trực tiếp sang thế hệ tiếp theo theo cơ chế *elitism*. Cơ chế này giúp bảo toàn các nghiệm tốt đã tìm được và tránh việc nghiệm tốt bị mất do lai ghép hoặc đột biến.

Nếu thuật toán không cải thiện sau một số thế hệ liên tiếp, biến *stagnant* tăng lên. Khi số thế hệ không cải thiện đạt ngưỡng, chương trình thực hiện restart một phần quần thể. Cụ thể, thuật toán giữ lại các cá thể elite, sau đó sinh lại phần còn lại của quần thể bằng các chromosome kiểu *hard_first* hoặc *random*. Cơ chế này giúp tăng lại độ đa dạng của quần thể khi quá trình tiến hóa bị đình trệ.

Cần lưu ý rằng cơ chế restart này không phải là tìm kiếm cục bộ. Thuật toán không chỉnh sửa trực tiếp lịch học bằng các thủ tục như *ejection* hoặc *destroy-repair*, mà chỉ tái tạo lại một phần quần thể để tiếp tục quá trình tiến hóa.

2.5.9 Kiểm tra tính hợp lệ của nghiệm

Sau khi thuật toán kết thúc, nghiệm tốt nhất được kiểm tra lại bằng hàm validate. Quá trình kiểm tra bao gồm:

- kiểm tra ID lớp, môn và giáo viên có nằm trong phạm vi hợp lệ hay không;
- kiểm tra lớp có thực sự yêu cầu môn học đó hay không;
- kiểm tra giáo viên có thể dạy môn học được phân công hay không;
- kiểm tra mỗi cặp lớp-môn không bị lặp;
- kiểm tra thời lượng môn học hợp lệ;
- kiểm tra kíp bắt đầu không vượt ngoài 60 slot;
- kiểm tra môn học không bị vắt qua buổi;
- kiểm tra không trùng lịch lớp;
- kiểm tra không trùng lịch giáo viên.

Chỉ các nghiệm vượt qua toàn bộ bước kiểm tra này mới được xem là nghiệm hợp lệ.

2.5.10 Sơ đồ luồng thuật toán

Quy trình tổng quát của thuật toán được mô tả như sau:

2.5.11 Thiết kế thí nghiệm

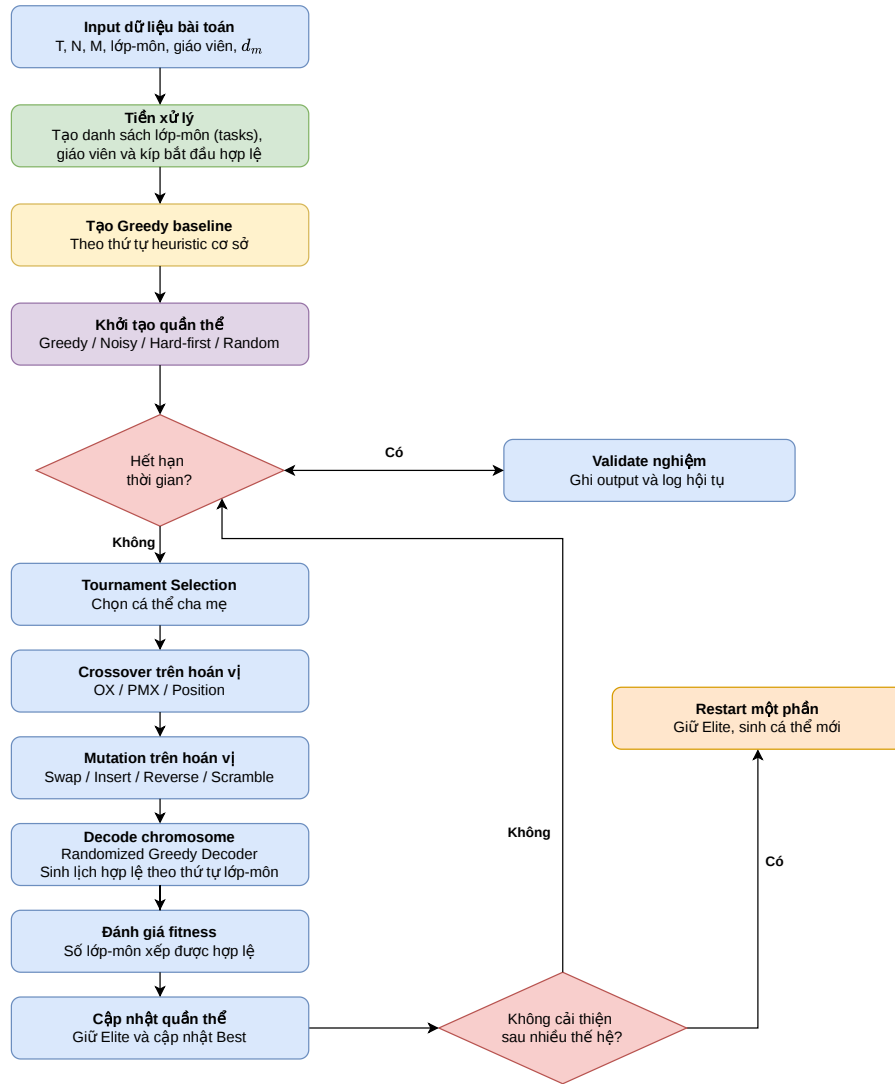
Để đánh giá ảnh hưởng của các thành phần trong thuật toán di truyền, nhóm thực hiện 8 cấu hình thí nghiệm. Các thí nghiệm được chia thành hai nhóm chính:

- Nhóm khởi tạo heuristic: quần thể ban đầu có sử dụng greedy baseline và các cá thể có định hướng heuristic.
- Nhóm khởi tạo random: quần thể ban đầu không đưa greedy baseline vào quần thể, nhằm quan sát rõ hơn khả năng tự hội tụ của thuật toán di truyền.

Các siêu tham số chung được giữ cố định trong phần lớn thí nghiệm:

$population_size = 40, elite = 5, crossover_rate = 0.85, tournament_k = 3, seed = 42$.

Danh sách thí nghiệm cụ thể như sau:



Hình 5: Sơ đồ hoạt động của thuật toán GA

Bảng 2: Các cấu hình thí nghiệm của thuật toán GA

STT	Tên thí nghiệm	Init	Crossover	Mutation rate	Mutation type
1	GA base OX + Mixed	Heuristic	OX	0.75	Mixed
2	GA low mutation	Heuristic	OX	0.40	Mixed
3	GA high mutation	Heuristic	OX	0.90	Mixed
4	GA PMX + Mixed	Heuristic	PMX	0.75	Mixed
5	Random base OX + Mixed	Random	OX	0.75	Mixed
6	Random low mutation	Random	OX	0.40	Mixed
7	Random high mutation	Random	OX	0.90	Mixed
8	Random PMX + Mixed	Random	PMX	0.75	Mixed

Mục đích của từng nhóm thí nghiệm như sau:

- So sánh **khởi tạo heuristic** và **khởi tạo random** để đánh giá vai trò của tri thức

heuristic trong nghiệm ban đầu.

- So sánh các mức **mutation rate** 0.40, 0.75 và 0.90 để đánh giá ảnh hưởng của cường độ đột biến.
- So sánh **OX crossover** và **PMX crossover** để đánh giá toán tử lai ghép phù hợp hơn với biểu diễn hoán vị của bài toán.
- Giữ cố định population size, elite, crossover rate, tournament size và seed để việc so sánh giữa các cấu hình công bằng hơn.

2.5.12 Nhận xét

Thuật toán di truyền trong bài làm không tìm kiếm trực tiếp trên toàn bộ không gian thời khóa biểu, mà tìm kiếm trên không gian thứ tự xếp các lớp-môn. Đây là cách tiếp cận phù hợp với bài toán vì thứ tự xếp lịch ảnh hưởng trực tiếp đến khả năng tìm được giáo viên và kíp học hợp lệ cho các lớp-môn khó.

Ưu điểm chính của phương pháp là chromosome gọn, các toán tử di truyền dễ triển khai và luôn bảo toàn được cấu trúc hoán vị. Bộ giải mã tham lam ngẫu nhiên hóa giúp chuyển chromosome thành thời khóa biểu hợp lệ, đồng thời tạo ra sự đa dạng trong nghiệm. Cơ chế elitism giúp giữ lại nghiệm tốt, còn restart một phần quần thể giúp thuật toán tránh bị đình trệ quá lâu.

Tuy nhiên, đây vẫn là một phương pháp heuristic/meta-heuristic nên không đảm bảo tìm được nghiệm tối ưu toàn cục. Chất lượng nghiệm phụ thuộc vào các siêu tham số như kích thước quần thể, số elite, tỷ lệ lai ghép, tỷ lệ đột biến, kiểu lai ghép, kiểu đột biến, chế độ khởi tạo và giới hạn thời gian chạy.

2.6 Thuật toán Hill Climbing Local Search (HCLS)

Thuật toán được đề xuất là sự kết hợp giữa kỹ thuật Tìm kiếm Cục bộ Leo đồi Ngẫu nhiên (Stochastic Hill Climbing Local Search - HCLS) và cơ chế Lựa chọn Toán tử Thích nghi (Adaptive Operator Selection - AOS). Thay vì sử dụng một toán tử cố định hoặc chọn ngẫu nhiên mù quáng, cấu trúc AOS cho phép hệ thống tự động "học" và điều chỉnh xác suất sử dụng các toán tử lân cận dựa trên hiệu quả thực tế của chúng trong quá trình tìm kiếm. Thuật toán hoạt động qua ba giai đoạn chính: Khởi tạo nghiệm cơ sở, Cải thiện nghiệm qua vòng lặp AOS-HCLS, và Dừng hệ thống khi hội tụ.

2.6.1 Hàm mục tiêu (Fitness Function)

Để thuật toán phân biệt được các trạng thái nghiệm, Hàm mục tiêu $F(S)$ được thiết kế như sau:

$$F(S) = W_1 \cdot |S| + W_2 \sum_{(n,m) \in S} (60 - u_{n,m}) \quad (6)$$

Trong đó:

- $|S|$: Kích thước của tập lời giải hiện tại, đại diện cho tổng số khối Lớp-Môn đã được sắp xếp lịch thành công.
- $u_{n,m}$: Kíp bắt đầu của môn học m tại lớp học n . Ví dụ: Nếu môn học 1 của lớp 3 được xếp vào kíp thứ 13 trong tuần, thì $u_{3,1} = 13$.

Trong quá trình thiết kế hàm Thích nghi (Fitness Function) cho giải thuật tìm kiếm cục bộ, chúng em thiết lập trọng số $W_1 = 1000$ nhằm ưu tiên tuyệt đối cho mục tiêu cốt lõi của bài toán gốc: tối đa hóa số lượng môn học được phân công. Trọng số phụ được chọn là $W_2 = 1$ nhằm tạo ra yêu cầu, khuyến khích thuật toán ưu tiên xếp các môn học bắt đầu ở các kíp càng sớm càng tốt (hệ số $u_{n,m}$ nhỏ).

Chiến lược này giúp chống phân mảnh không gian thời khóa biểu, dọn dẹp các kíp học lên đầu tuần và tạo ra các khoảng trống thời gian lớn ở giai đoạn cuối tuần. Nhờ cấu trúc đặc này, hệ thống có nhiều dư địa trống (Neighborhood space) để dễ dàng chen thêm các môn học khó ở những vòng lặp tìm kiếm tiếp theo.

2.6.2 Khởi tạo Lời giải

Nghiem cơ sở của hệ thống được xây dựng dựa trên phương pháp sau. Quá trình này bắt đầu bằng việc đánh giá và sắp xếp toàn bộ các yêu cầu khối Lớp-Môn theo thứ tự độ khó giảm dần, dựa trên hệ thức:

$$D = d_m \times 100 - |T_m| \quad (7)$$

Trong đó, d_m là số kíp học liên tiếp cần thiết của môn m , và $|T_m|$ là lực lượng của tập giáo viên có chuyên môn giảng dạy môn đó.

Hệ số khuếch đại 100 được đưa vào nhằm thiết lập ưu tiên. Việc để trọng số tập trung vào d_m , bọn em mong muốn thời lượng môn học đóng vai trò là tiêu chí phân loại chính, ép thuật toán phải ưu tiên lấp đầy các môn học kéo dài nhiều tiết trong lúc quỹ thời gian còn trống trải. Tiêu chí số lượng giáo viên $|T_m|$ đóng vai trò là tiêu chí phụ để ưu tiên các môn học thiếu hụt nhân lực khi có nhiều môn cùng độ dài.

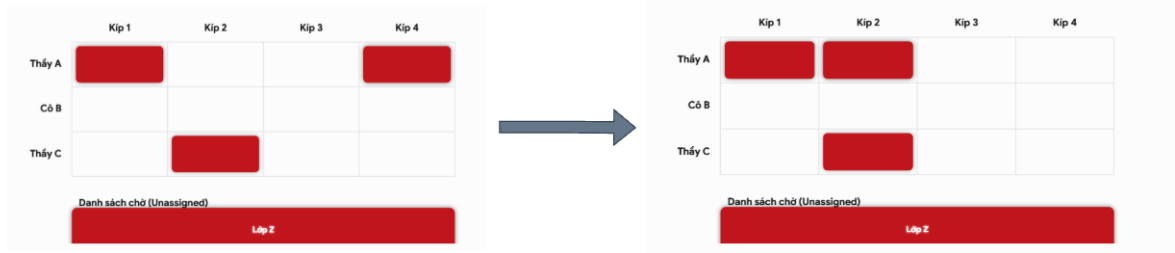
Dựa trên danh sách đã sắp xếp, hệ thống tuần tự duyệt qua từng khối Lớp-Môn và gán ngẫu nhiên vào một không gian phân công (gồm kíp bắt đầu và giáo viên phụ trách) hoàn toàn hợp lệ, không vi phạm ràng buộc.

Để ngăn chặn hiện tượng bùng nổ thời gian thực thi ở các bộ dữ liệu quy mô lớn ($N \geq 500$), thuật toán khởi tạo được thiết kế duyệt một chiều. Khi gặp một khối Lớp-Môn không thể tìm được vị trí khả thi do dè lịch, hệ thống sẽ ngay lập tức bỏ qua và đẩy yêu cầu này vào danh sách chờ (*Unassigned_List*). Chiến lược này kiểm soát độ phức tạp thời gian của pha khởi tạo ở mức $O(E \cdot |U| \cdot |T_{max}|)$ (với E là tổng số lượng Lớp-Môn), đảm bảo việc sinh ra nghiệm cơ sở luôn diễn ra trong thời gian tức thời, nhường lại tác vụ giải quyết xung đột phức tạp cho giai đoạn Tìm kiếm Cục bộ (Local Search) phía sau.

2.6.3 Các Toán tử Lân cận

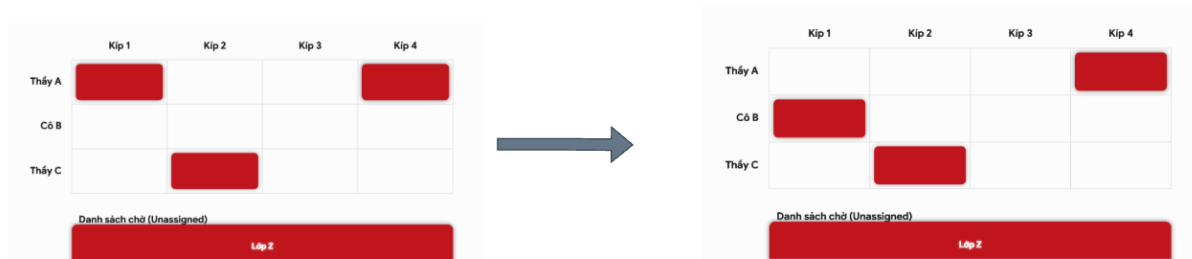
Tại mỗi bước lặp, trạng thái lân cận được sinh ra thông qua một trong ba toán tử:

- **Shift (Trượt kíp):** Chọn ngẫu nhiên 1 môn trong lịch, giữ nguyên giáo viên, dời sang một kíp thời gian trống khác.



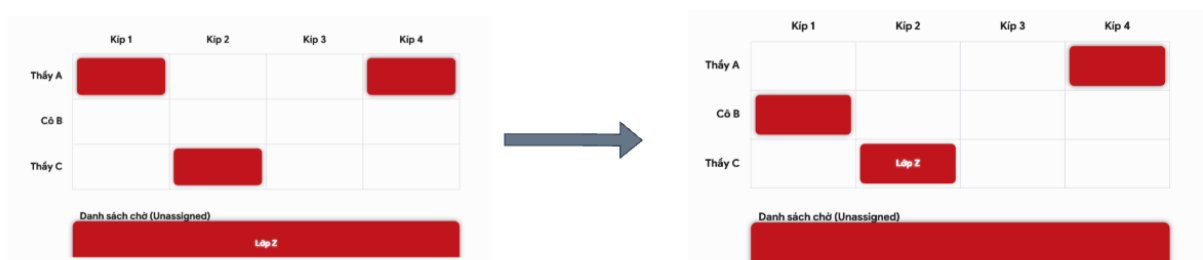
Hình 6: Sơ đồ thuật toán tham lam

- **Swap (Đổi giáo viên):** Giữ nguyên kíp học, chuyển môn đó cho một giáo viên có chuyên môn khác đang rảnh rỗi.



Hình 7: Sơ đồ thuật toán tham lam

- **Kick (Đẩy ngã):** Rút 1 môn từ *Unassigned_List*, chen ép vào một vị trí ngẫu nhiên. Các môn học đang chiếm dụng vị trí này bị "đá" ra khỏi lịch. Đây là toán tử phá hủy nhằm thoát khỏi tối ưu cục bộ.



Hình 8: Sơ đồ thuật toán tham lam

2.6.4 Vòng quay Adaptive và chọn điểm

Mỗi toán tử được gán một trọng số $W(op)$ khởi điểm bằng 100.0. Xác suất được gọi của một toán tử tỷ lệ thuận với trọng số của nó. Sau khi áp dụng, hệ thống cấp điểm thưởng

(*Reward*):

- +50 điểm: Sinh ra Kỷ lục toàn cục mới ($F_{new} > F_{best}$).
- +20 điểm: Nghiệm tốt hơn hiện tại nhưng chưa vượt kỷ lục ($F_{best} \geq F_{new} > F_{current}$).
- +5 điểm: Nghiệm ngang bằng hiện tại (Sideways Move).
- 0 điểm: Bị từ chối do nghiệm tệ hơn.

Trọng số được cập nhật theo trung bình trượt với hệ số quên $\rho = 0.2$:

$$W_{t+1}(op) = \max(5.0, (1 - \rho)W_t(op) + \rho \cdot Reward) \quad (8)$$

2.6.5 Cơ chế dừng

Thuật toán kết thúc khi đạt một trong ba điều kiện:

- Đạt tối ưu tuyệt đối (100%)
- Đạt giới hạn số vòng lặp tối đa ($Max_Iterations$)
- Trì trệ cục bộ (Không cải thiện kỷ lục sau $Max_No_Improve$ vòng lặp).

2.7 Adaptive Large Neighbourhood Search (ALNS)

Adaptive Large Neighbourhood Search (ALNS) là một hướng tiếp cận meta-heuristic dựa trên nguyên lý phá hủy và tái xây dựng nghiệm. Thay vì chỉ thay đổi cục bộ một lớp-môn như các phép trượt kíp hoặc đổi giáo viên đơn lẻ, ALNS chủ động loại bỏ một phần tương đối lớn của thời khóa biểu hiện tại, sau đó cố gắng chèn lại các lớp-môn chưa được xếp vào những vị trí khả thi mới. Nhờ vậy, thuật toán có khả năng thoát khỏi các tối ưu cục bộ tốt hơn so với tìm kiếm lân cận nhỏ.

Trong triển khai của nhóm, hàm mục tiêu được định nghĩa đơn giản là số lượng lớp-môn được xếp lịch thành công:

$$F(S) = |S|, \quad (9)$$

trong đó S là tập các phân công hợp lệ hiện có trong thời khóa biểu. Mỗi phân công được biểu diễn bởi bộ (c, m, u, t) , tương ứng với lớp c , môn m , kíp bắt đầu u và giáo viên t . Thuật toán vẫn đảm bảo các ràng buộc cốt lõi của bài toán, bao gồm không trùng lịch lớp, không trùng lịch giáo viên và không để môn học vắt qua ranh giới buổi học.

2.7.1 Khởi tạo nghiệm

Ban đầu, thuật toán xây dựng một nghiệm khả thi bằng thủ tục chèn tham lam ngẫu nhiên. Danh sách các yêu cầu lớp-môn được xáo trộn ngẫu nhiên. Với mỗi yêu cầu (c, m) , thuật toán xáo trộn danh sách giáo viên có thể dạy môn m và danh sách các kíp bắt đầu hợp lệ của môn đó. Sau đó, thủ tục **fast_insertion** tìm cặp (u, t) đầu tiên sao cho lớp

c và giáo viên t đều rảnh trong toàn bộ khoảng thời gian từ u đến $u + d_m - 1$. Nếu tìm được vị trí hợp lệ, yêu cầu được đưa vào lịch; ngược lại, yêu cầu tạm thời bị bỏ qua. Để tăng tốc kiểm tra xung đột, thuật toán sử dụng hai ma trận nhị phân:

- `class_busy` $[c, u]$: nhận giá trị 1 nếu lớp c bận tại kíp u ;
- `teacher_busy` $[t, u]$: nhận giá trị 1 nếu giáo viên t bận tại kíp u .

Cách biểu diễn này giúp kiểm tra tính khả thi của một phép chèn bằng thao tác tra cứu trực tiếp trên mảng, thay vì phải duyệt lại toàn bộ lịch hiện có.

2.7.2 Pha phá hủy và tái xây dựng

Sau khi có nghiệm ban đầu, ALNS lặp lại hai bước chính:

1. **Destroy:** Chọn ngẫu nhiên một số lớp–môn đang được xếp lịch và loại bỏ chúng khỏi nghiệm hiện tại. Số lượng phần tử bị loại bỏ được lấy ngẫu nhiên trong khoảng từ 1 đến $\max(1, |S|/r)$, với r là tham số *destroy ratio*. Khi một lớp–môn bị loại bỏ, trạng thái bận của lớp và giáo viên tương ứng cũng được cập nhật lại về rảnh.
2. **Repair:** Tạo danh sách tất cả các yêu cầu lớp–môn chưa có trong nghiệm sau pha phá hủy, xáo trộn danh sách này và lần lượt thử chèn lại bằng thủ tục **fast_insertion**. Nếu tìm được giáo viên và kíp bắt đầu hợp lệ, lớp–môn được thêm trở lại lịch.

Quá trình phá hủy giúp thuật toán giải phóng các vùng thời gian đã bị lấp đầy kém hiệu quả, còn quá trình tái xây dựng cho phép chèn lại cả các lớp–môn vừa bị loại bỏ lẫn các lớp–môn chưa từng được xếp thành công. Đây là cơ chế chính giúp ALNS mở rộng không gian tìm kiếm.

2.7.3 Cơ chế chấp nhận nghiệm

Để tránh bị kẹt tại tối ưu cục bộ, thuật toán sử dụng cơ chế chấp nhận theo mô phỏng luyện kim (Simulated Annealing). Gọi F_{cur} là fitness của nghiệm hiện tại và F_{new} là fitness của nghiệm lân cận sau pha destroy–repair. Độ chênh lệch được tính bởi:

$$\Delta = F_{new} - F_{cur}. \quad (10)$$

Nếu $\Delta > 0$, nghiệm mới được chấp nhận ngay vì cải thiện số lớp–môn được xếp. Nếu $\Delta \leq 0$, nghiệm mới vẫn có thể được chấp nhận với xác suất:

$$P = \exp\left(\frac{\Delta}{Temp}\right), \quad (11)$$

trong đó $Temp$ là nhiệt độ hiện tại. Sau mỗi vòng lặp, nhiệt độ được làm nguội theo công thức:

$$Temp \leftarrow \alpha \cdot Temp, \quad (12)$$

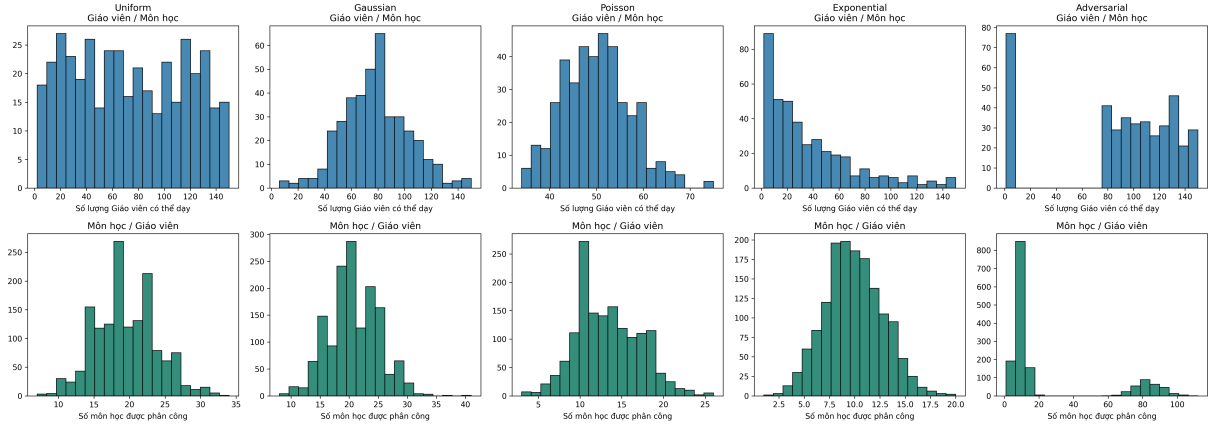
với α là hệ số làm nguội. Khi nhiệt độ còn cao, thuật toán có khả năng chấp nhận một số nghiệm kém hơn để khám phá không gian nghiệm. Khi nhiệt độ giảm dần, thuật toán trở nên tham lam hơn và ưu tiên giữ các nghiệm tốt.

Bảng 3: Siêu tham số chính của thuật toán ALNS

Siêu tham số	Giá trị	Ý nghĩa
$Temp_0$	100.0	Nhiệt độ ban đầu trong cơ chế chấp nhận SA
α	0.99	Hệ số làm nguội sau mỗi vòng lặp
$Iter_{max}$	2,000	Số vòng lặp destroy–repair tối đa
r	3	Tỷ lệ phá hủy, dùng để giới hạn số phần tử bị loại khỏi lịch
$F(S)$	$ S $	Hàm mục tiêu: số lớp–môn được xếp thành công

3 THỰC NGHIỆM

Chương này trình bày kết quả và các tài nguyên cần thiết cho bài tập lớn này. Bao gồm về dữ liệu sử dụng, thiết bị thực nghiệm và các kết quả thực nghiệm cho bài này. Nội dung chương trình được lưu ở https://github.com/HUST-8xbet/Class_Course_Teaching_Assignment_and_TimeTabling



Hình 9: Phân phối dữ liệu với bài toán $N = 1000$, $T = 1500$, $M = 400$

3.1 Dữ liệu

Nhóm chúng em thực hiện tạo bộ dữ liệu có kích thước 60, gồm 10 bài của hustack và 50 bài tự sinh ra với các bộ (N, M, T) khác nhau với tập hợp môn bắt buộc của mỗi lớp được khởi tạo có điều kiện và số giáo viên dạy được môn học tuân theo các phân phối như: Uniform(Đều), Gaussian(Chuẩn), Exponential(Mũ), Poisson và Adversarial

Bảng 4: Cấu hình hệ thống tham số và các ràng buộc trong quy trình sinh dữ liệu

Parameter	Values
Số lượng lớp học (N)	20, 50, 80, 100, 150, 200, 300, 500, 800, 1000
Số lượng giáo viên (T)	$1.5 \times N$ (Quy mô tỷ lệ tuyến tính)
Số lượng môn học (M)	$\max(10, 0.4 \times N)$
Thời lượng tiết học (d_m)	2, 3, 4 kíp (Ngẫu nhiên cho mỗi môn m)
Dạng phân phối dữ liệu (Distribution)	Uniform, Gaussian, Poisson, Exponential, Adversarial
Số lượng GV tối đa/môn ($ T_m $)	Từ 2 đến $\max(3, \lfloor 0.1 \times T \rfloor)$ giáo viên
Số tiết học tối đa của một lớp	≤ 45 kíp học
Số tiết học tối thiểu của một lớp	≥ 35 kíp học
Số lượng môn học tối đa của một lớp	≤ 12 môn học
Số tiết giảng dạy tối đa của một GV	≤ 45 kíp học (Áp dụng cho mọi giáo viên t)

Để đa dạng hóa dữ liệu kiểm thử, ban đầu bọn em mô phỏng các dạng phân phối phổ biến bao gồm Uniform, Gaussian, Poisson và Exponential bằng cách sử dụng thư viện

ngẫu nhiên. Tuy nhiên, kết quả phân tích mật độ ràng buộc ở hình[2] chỉ ra một hiện tượng đáng lưu ý: mặc dù cấu hình số lượng giáo viên khả dụng cho mỗi môn học tuân theo các phân phối khác nhau, nhưng phân phối số lượng môn học được phân công trên mỗi giáo viên hầu hết đều hội tụ về dạng phân phối chuẩn (phân phối Gauss). Chúng em cho rằng hiện tượng này là hệ quả trực tiếp của Định lý Giới hạn Trung tâm (Central Limit Theorem - CLT) [?].

Theo lý thuyết xác suất, Định lý Giới hạn Trung tâm phát biểu rằng khi một biến số là tổng của một số lượng lớn các biến ngẫu nhiên độc lập, phân phối của nó sẽ có xu hướng tiến về dạng hình chuông chuẩn. Trong bài toán của chúng tôi, tổng số môn học mà một giáo viên có thể đảm nhiệm bản chất là sự tích lũy của nhiều sự kiện lựa chọn độc lập giữa lớp-môn và giáo viên. Quá trình lấy mẫu ngẫu nhiên lũy tiến này trên quy mô hàng trăm môn học đã khiến mật độ phân công giáo viên tự động hội tụ về một đường cong chuẩn, vô tình làm giảm tính đa dạng và triệt tiêu độ khó thực tế của dữ liệu kiểm thử. Do đó, bọn em đã chủ động thiết kế một kịch bản phân phối đặc biệt mang tên "Adversarial" nhằm tái hiện hiện tượng khan hiếm nhân lực chuyên môn sâu và tạo ra một cấu trúc phân phối hoàn toàn dị biệt. Quy trình này thiết lập một cơ chế cô lập: Bọn em tách biệt một đến hai "Giáo viên cốt lõi" chỉ đảm nhiệm duy nhất một "Môn học cốt lõi", đồng thời loại bỏ hoàn toàn khả năng giảng dạy môn học này đối với tất cả các giáo viên khác trong hệ thống. Ngược lại, nhu cầu của các lớp học được phân bổ theo hai kịch bản: các lớp thông thường sẽ bốc thăm ngẫu nhiên các môn học từ tập phổ thông, trong khi một số lượng lớp nhất định (*Special Classes*) được tính toán chính xác để ép buộc phải học môn học cốt lõi này. Cấu trúc bẫy này được thiết kế nhằm tạo ra sự mất cân đối cực hạn trong ma trận ràng buộc, trực tiếp dồn tải trọng đỉnh và các xung đột lịch trình cục bộ để thách thức giới hạn xử lý của các thuật toán cũng như các bộ giải toán học.

3.2 Môi trường thực nghiệm

Bảng 4 cung cấp về môi trường bọn em sẽ sử dụng trong bài toán này.

Bảng 5: Cấu hình môi trường phần cứng và phần mềm thực nghiệm

Thông số kỹ thuật	Chi tiết cấu hình
Bộ vi xử lý (CPU)	Intel(R) Xeon(R) CPU @ 2.00GHz (2 cores)
Bộ nhớ truy cập ngẫu nhiên (RAM)	16 GB
Hệ điều hành (OS)	Ubuntu 22.04 LTS (64-bit)
Nền tảng thực nghiệm (Platform)	Kaggle Notebooks
Môi trường thực thi (Runtime)	Python 3.11.7

4 KẾT QUẢ THỰC NGHIỆM

Kết quả thực nghiệm được lưu ở trong phụ lục, ở đây khi để so sánh khả năng tối ưu của các thuật toán Heuristic, Metaheuristic, chúng em lấy benchmark là những bài mà các bộ giải tối ưu giải được OPTIMUM, còn với những bài bộ giải không giải được, chúng em thực hiện so sánh với nhau kết quả tối ưu của chúng

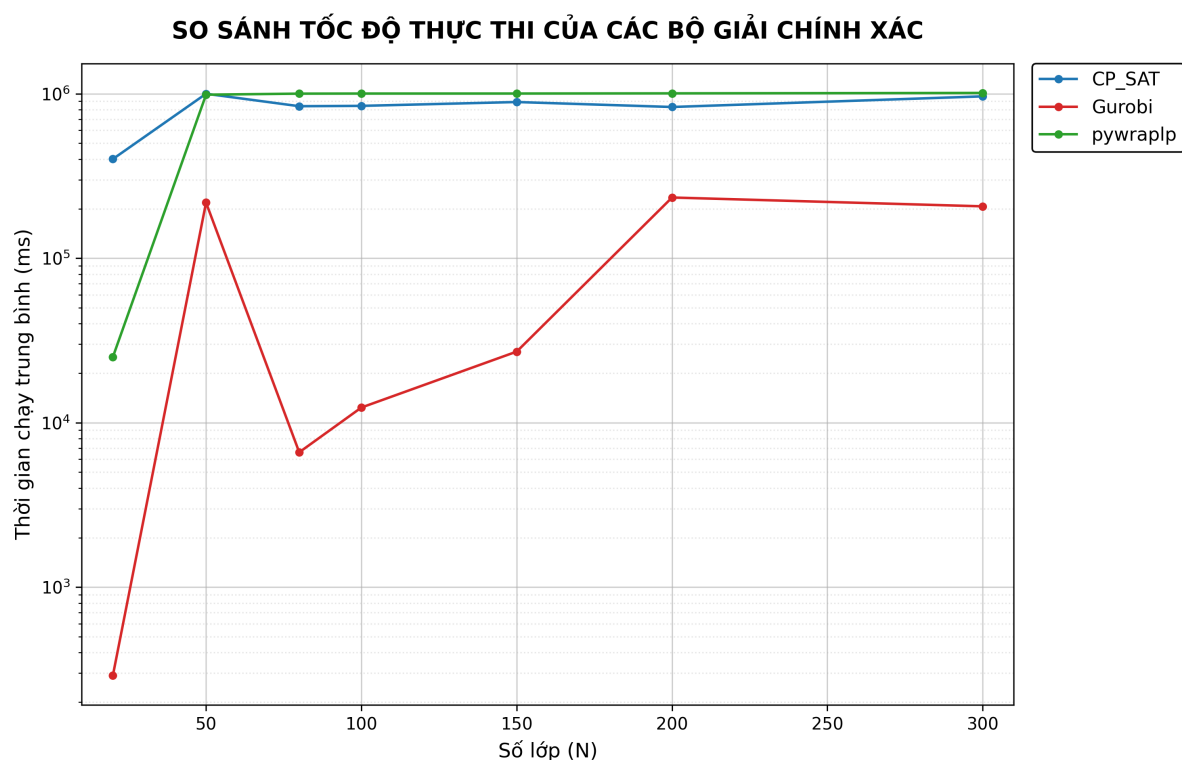
4.1 Bộ giải chính xác nguyên (ILP)

Bảng 6: Tổng hợp hiệu năng của các bộ giải chính xác trên 60 bài toán

Bộ giải (Solver)	Số bài giải thành công / 60	Số bài đạt tối ưu / 60
CP-SAT	58	18
Gurobi	50	47
pywraplp	45	16

Đây là bảng so sánh giữa 3 thuật toán khi thực hiện giải trên bộ test bọn em sinh ra với việc giới hạn chúng chạy trong 1000 giây. Từ kết quả thống kê tại bảng trên, có thể rút ra những đánh giá quan trọng về đặc tính và hiệu năng của ba bộ giải toán học khi xử lý bài toán lập lịch:

- **Khả năng tìm kiếm nghiệm khả thi (Tính linh hoạt):** Bộ giải CP-SAT thể hiện khả năng tốt nhất trong việc số bài có thể giải được là lớn nhất, cụ thể giải quyết thành công 58/60 bài toán thử nghiệm. Điều này do sức mạnh cốt lõi của Constraint Programming kết hợp SAT engine: thuật toán ưu tiên cắt tĩa logic để nhanh chóng tìm ra các nghiệm hợp lệ (Feasible), giúp hệ thống hiếm khi rơi vào tình trạng cạn kiệt thời gian mà không thu được kết quả nào.
- **Khả năng chứng minh tính tối ưu:** Trái ngược với CP-SAT, Gurobi khẳng định vị thế dẫn đầu về số bài tối ưu. Mặc dù số lượng bài giải thành công thấp hơn (50/60 bài) do rào cản về thời gian tính toán, nhưng có đến 47 bài (chiếm 94% số bài giải được) được Gurobi chứng minh toán học là nghiệm tối ưu tuyệt đối (Optimum). Sự chênh lệch này phản ánh rõ nét bản chất của thuật toán Nhánh và Cận (Branch & Bound) kết hợp với các mặt cắt (Cutting Planes) bên trong Gurobi, cho phép nó vét cạn không gian nhánh hiệu quả hơn hẳn khi so với số lượng tối ưu khiêm tốn của CP-SAT (18 bài) và pywraplp (16 bài).



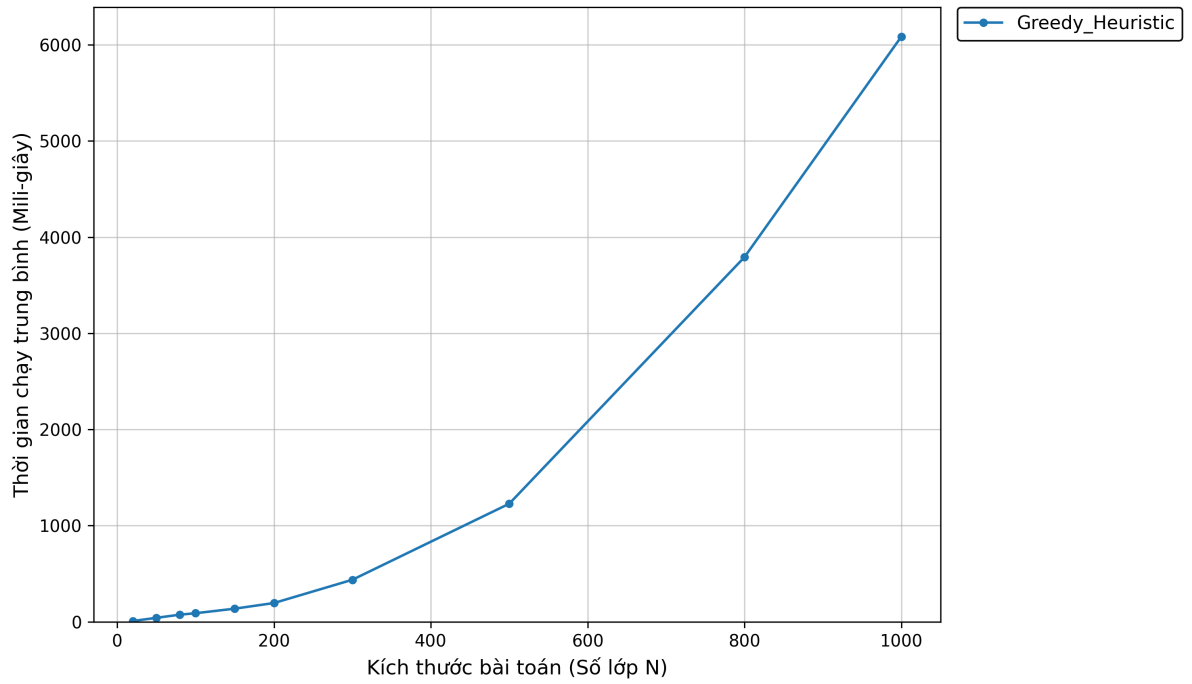
Hình 10: Biểu đồ thời gian chạy của các bộ giải qua các kích thước khác nhau của dữ liệu

Dựa vào biểu đồ so sánh tốc độ thực thi, ta có thể quan sát rõ sự biến thiên về thời gian chạy của ba bộ giải chính xác khi mở rộng kích thước bài toán. Kết quả cho thấy Gurobi (đường nét đứt màu đỏ) thể hiện tốc độ vượt trội và khả năng mở rộng (scalability) tốt nhất. Ngay từ các mốc dữ liệu nhỏ và trung bình (từ $N = 50$ trở đi), hiệu năng của CP-SAT và pywraplp đã suy giảm nhanh chóng và chạm ngưỡng giới hạn thời gian (Time Limit) là 1000 giây (tương đương 1.000.000 ms), thể hiện qua việc đường biểu diễn lập tức đi ngang. Trong khi đó, Gurobi vẫn duy trì được thời gian tính toán thấp hơn đáng kể xuyên suốt các mốc thử nghiệm, minh chứng cho khả năng xử lý hiệu quả cấu trúc toán học của bài toán này."

4.2 Thuật toán tham lam

Trong thí nghiệm này, thuật toán tham lam được dùng để thể hiện khả năng sinh ra lời giải tốt trong thời gian ngắn.

TỐC ĐỘ THỰC THI CỦA THUẬT TOÁN THAM LAM (GREEDY)



Hình 11: Biểu đồ thời gian chạy của thuật toán tham lam qua các kích thước khác nhau của dữ liệu

Qua bức ảnh trên, ta thấy rằng tốc độ thuật toán chạy thay đổi tăng dần qua từng kích cỡ, song thời gian tối đa là khoảng 6000ms (6 giây), thời gian đủ nhỏ và vẫn có thể đạt được tối ưu 16/60 bài (xem Phụ lục)

4.3 Thuật toán HCLS

Bảng 7 tóm tắt các siêu tham số chính được sử dụng trong triển khai AOS-HCLS. Các giá trị này được lấy trực tiếp từ cấu hình thực nghiệm của chương trình.

Bảng 7: Siêu tham số chính của thuật toán AOS-HCLS

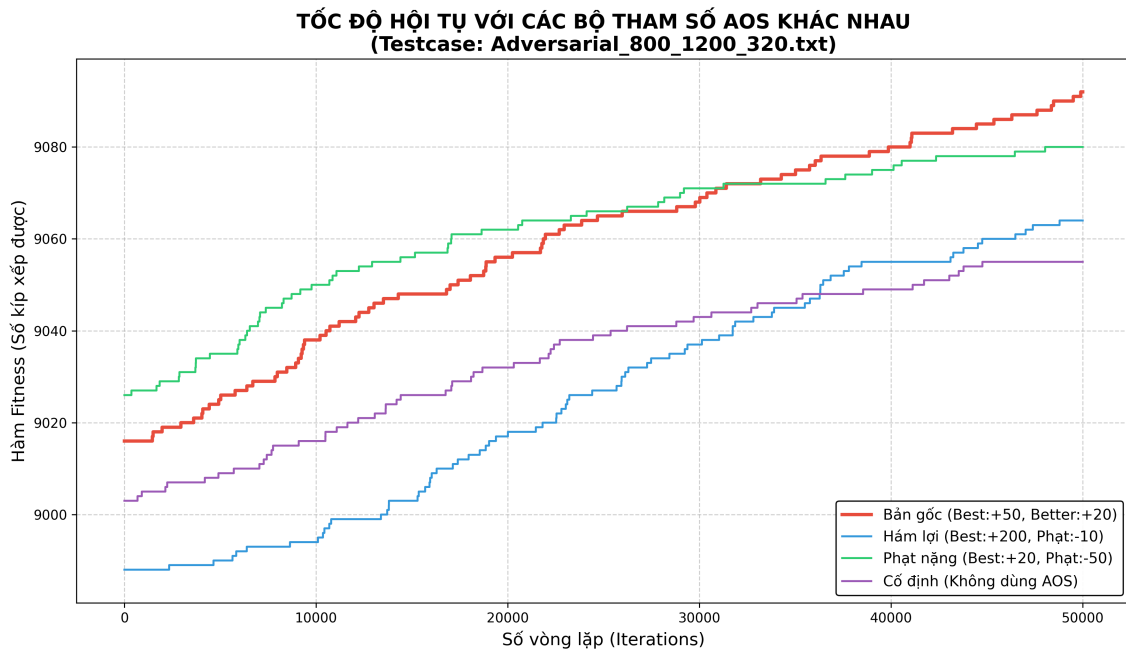
Siêu tham số	Giá trị	Ý nghĩa
$Iter_{max}$	50,000	Số vòng lặp tìm kiếm tối đa
Nil_{max}	3,000	Số vòng lặp trì trệ tối đa trước khi dừng
ρ	0.2	Hệ số quên trong cập nhật trọng số toán tử AOS
$W_0(Shift)$	100.0	Trọng số ban đầu của toán tử trượt kịp
$W_0(Swap)$	100.0	Trọng số ban đầu của toán tử đổi giáo viên
$W_0(Kick)$	100.0	Trọng số ban đầu của toán tử đẩy/chèn lại lịch
W_{min}	5.0	Ngưỡng dưới của trọng số toán tử
$Reward_{best}$	50	Điểm thưởng khi tạo nghiệm tốt nhất mới
$Reward_{improve}$	20	Điểm thưởng khi cải thiện nghiệm hiện tại
$Reward_{equal}$	5	Điểm thưởng cho bước đi ngang bằng nghiệm hiện tại
$Reward_{reject}$	0	Điểm thưởng khi toán tử không cải thiện hoặc bị từ chối
$K_{conflict}$	2	Số xung đột tối đa cho phép khi áp dụng toán tử Kick
$W_{t+1}(op)$	$\max(5, (1 - \rho)W_t(op) + \rho R)$	Công thức cập nhật trọng số toán tử

Trong thí nghiệm HCLS, nhóm khảo sát ảnh hưởng của hệ số thích nghi ρ trong cơ chế cập nhật trọng số toán tử. Bảng 8 trình bày giá trị mục tiêu trung bình và thời gian chạy trung bình theo từng kích thước dữ liệu. Với mỗi dòng, giá trị mục tiêu tốt nhất và thời gian chạy nhỏ nhất được in đậm.

Bảng 8: Ảnh hưởng của hệ số ρ đến hiệu năng thuật toán HCLS

Kích thước N	$\rho = 0.1$		$\rho = 0.2$		$\rho = 0.5$		$\rho = 0.9$	
	Avg. Obj.	Avg. Time (ms)	Avg. Obj.	Avg. Time (ms)	Avg. Obj.	Avg. Time (ms)	Avg. Obj.	Avg. Time (ms)
200	2320.6	19,685.10	2321.0	19,730.80	2321.6	19,984.64	2320.0	20,231.48
800	9448.0	91,765.04	9448.4	92,695.98	9443.6	95,036.81	9446.0	96,706.33
1000	11826.0	123,869.98	11826.8	125,810.21	11825.8	128,750.68	11829.0	133,061.19

Kết quả cho thấy $\rho = 0.2$ và $\rho = 0.9$ thường đạt giá trị mục tiêu trung bình cao nhất trên các bộ dữ liệu lớn, trong khi $\rho = 0.1$ cho thời gian chạy thấp nhất ở cả ba kích thước. Điều này phản ánh sự đánh đổi giữa mức độ thích nghi của thuật toán và chi phí tìm kiếm: hệ số ρ lớn giúp trọng số toán tử phản ứng nhanh hơn với phần thưởng mới, nhưng có thể làm tăng thời gian thực thi trung bình.



Hình 12: So sánh ảnh hưởng của các cấu hình tham số AOS-HCLS theo kích thước dữ liệu

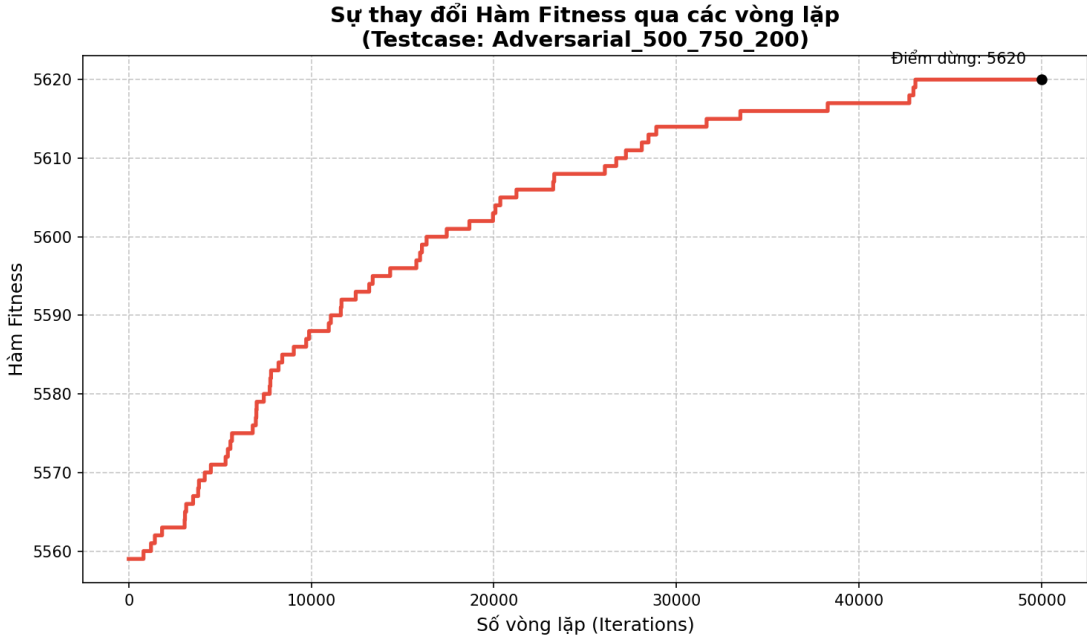
Hình 12 trực quan hóa sự khác biệt giữa các cấu hình tham số của AOS-HCLS. Biểu đồ giúp quan sát đồng thời chất lượng nghiệm và xu hướng thời gian chạy khi kích thước dữ liệu tăng, từ đó hỗ trợ lựa chọn cấu hình cân bằng giữa hiệu quả tối ưu và chi phí tính toán.

Bên cạnh đó, nhóm cũng đánh giá đóng góp của từng toán tử lân cận và cơ chế chọn toán tử thích nghi. Bảng 9 so sánh nghiệm khởi tạo, các biến thể chỉ dùng một toán tử, HCLS ngẫu nhiên và AOS-HCLS. Trong mỗi dòng, giá trị mục tiêu cao nhất và thời gian chạy nhỏ nhất trong các biến thể có thực hiện tìm kiếm được in đậm.

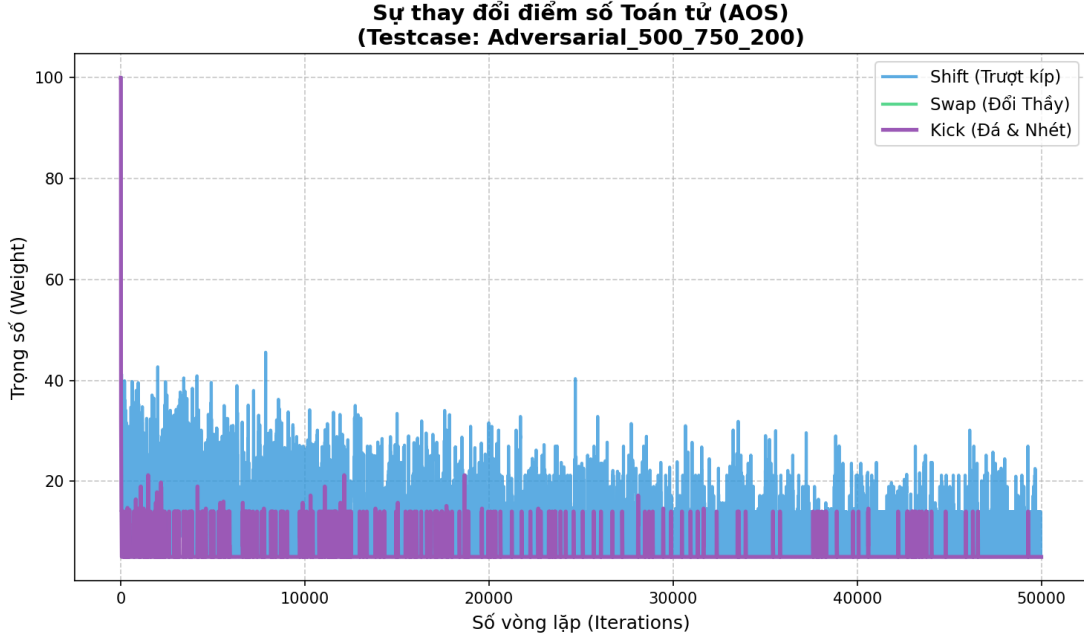
Bảng 9: So sánh các biến thể toán tử trong thuật toán HCLS

Kích thước N	Only Init		Only Shift		Only Swap		Only Kick		Random HCLS		AOS-HCLS	
	Avg. Obj.	Time (ms)	Avg. Obj.	Time (ms)	Avg. Obj.	Time (ms)	Avg. Obj.	Time (ms)	Avg. Obj.	Time (ms)	Avg. Obj.	Time (ms)
200	2311.0	0.00	2311.0	20,382.88	2311.0	3,428.15	2316.0	6,288.95	2319.4	32,251.24	2321.0	31,317.99
800	9433.0	0.00	9433.0	109,130.53	9433.0	17,494.71	9444.0	30,132.67	9448.2	178,034.36	9448.4	163,132.05
1000	11815.6	0.00	11815.6	158,312.64	11815.6	23,775.25	11824.2	45,131.82	11827.2	249,191.90	11826.8	227,024.76

Kết quả cho thấy toán tử *Kick* đóng góp rõ rệt hơn so với *Shift* và *Swap* khi xét riêng lẻ, do có khả năng phá vỡ cấu trúc nghiệm hiện tại để chen thêm các lớp-môn chưa được xếp. AOS-HCLS đạt nghiệm tốt nhất ở kích thước $N = 800$, trong khi Random HCLS nhỉnh hơn ở $N = 1000$; điều này cho thấy cơ chế thích nghi giúp ổn định chất lượng nghiệm, nhưng vẫn tồn tại sự đánh đổi với thời gian chạy khi bài toán lớn hơn.



Hình 13: Diễn biến giá trị fitness của thuật toán AOS-HCLS trên bộ dữ liệu Adversarial_500_750_200



Hình 14: Sự thay đổi trọng số các toán tử Shift, Swap và Kick trong AOS-HCLS trên bộ dữ liệu Adversarial_500_750_200

Hai biểu đồ trên minh họa quá trình hội tụ của AOS-HCLS trên một bộ dữ liệu có cấu trúc khó. Hình 13 cho thấy sự thay đổi của giá trị fitness theo thời gian, trong khi Hình 14 phản ánh cách cơ chế AOS điều chỉnh mức ưu tiên của từng toán tử lân cận trong quá trình tìm kiếm. Có thể thấy được rằng qua thời gian, thuật toán ưu tiên sử dụng hàm local search Shift, còn Kick được dùng trong một số vòng lặp để thoát khỏi tối ưu cục bộ.

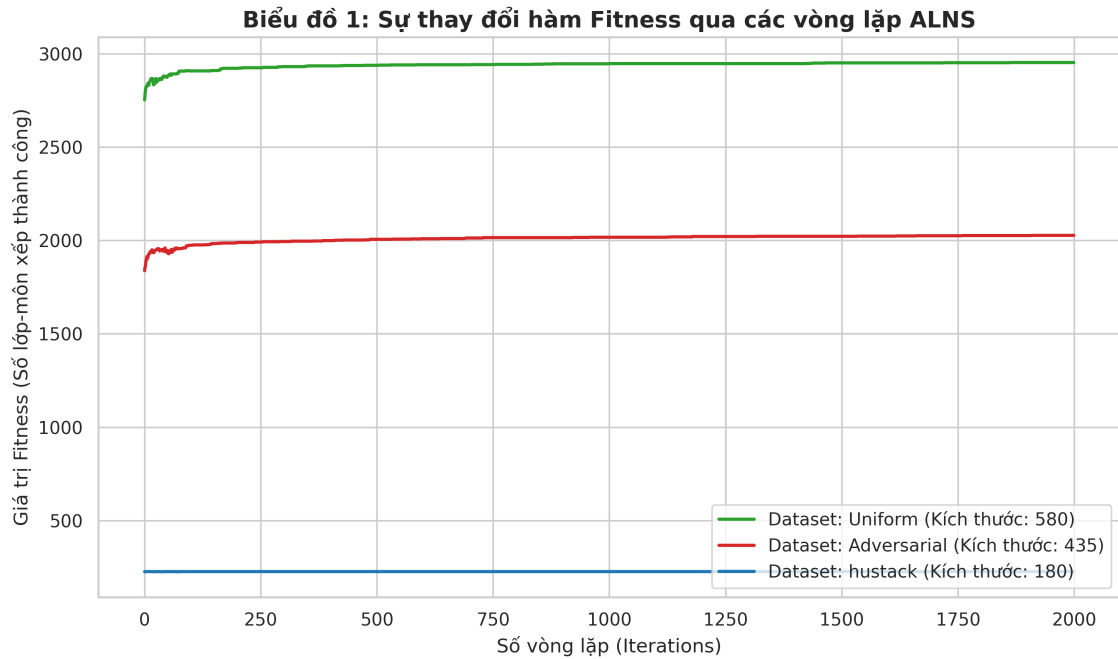
4.4 Thuật toán ALNS

Thuật toán ALNS được triển khai theo hướng phá hủy–tái xây dựng nghiệm. Trước hết, thuật toán sinh một lịch ban đầu bằng cơ chế chèn tham lam ngẫu nhiên. Ở mỗi vòng lặp, một phần các lớp–môn đã được xếp sẽ bị loại bỏ khỏi lịch hiện tại, sau đó thuật toán thử chèn lại toàn bộ các lớp–môn chưa được xếp bằng thủ tục **fast_insertion**. Nghiệm mới được chấp nhận nếu cải thiện giá trị mục tiêu, hoặc được chấp nhận với xác suất mô phỏng luyện kim nhằm tránh mắc kẹt tại tối ưu cục bộ.

Bảng 10: Siêu tham số chính của thuật toán ALNS

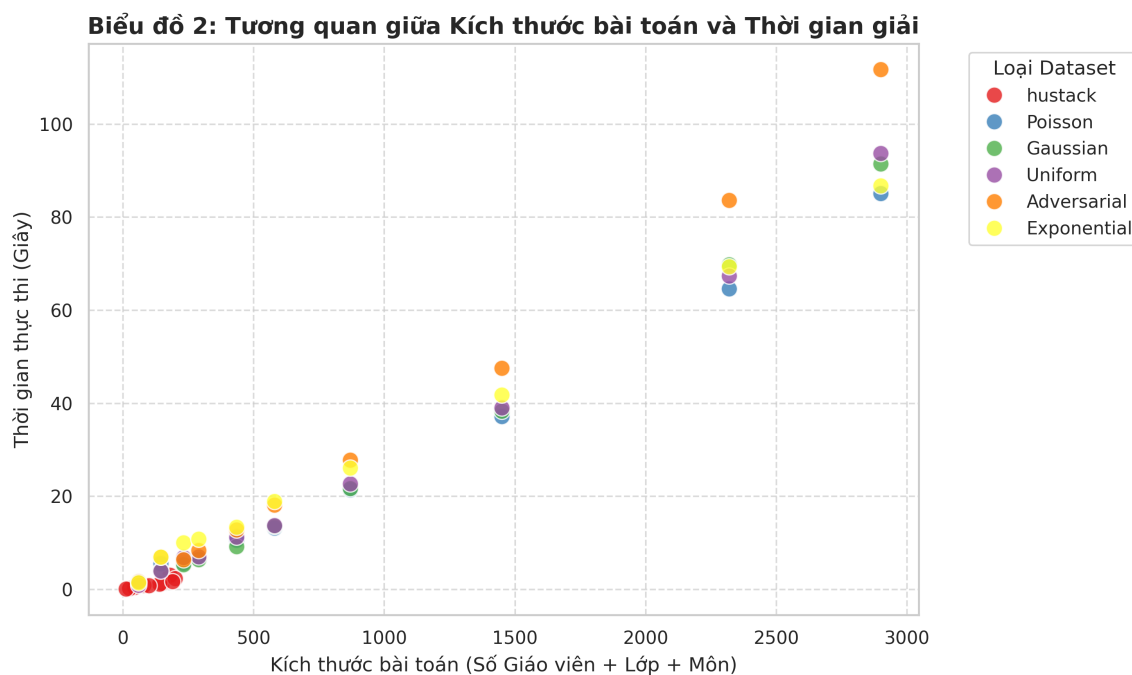
Siêu tham số	Giá trị	Ý nghĩa
$Temp_0$	100.0	Nhiệt độ ban đầu trong cơ chế chấp nhận nghiệm kiểu Simulated Annealing
α	0.99	Hệ số làm nguội sau mỗi vòng lặp, với $Temp \leftarrow \alpha \cdot Temp$
$Iter_{max}$	2000	Số vòng lặp phá hủy–tái xây dựng tối đa
$destroy_ratio$	3	Tỉ lệ kiểm soát số lớp–môn bị loại bỏ khỏi nghiệm hiện tại
n_{remove}	$[1, \lfloor Assigned / destroy_ratio \rfloor]$	Khoảng số lớp–môn bị phá hủy ngẫu nhiên ở mỗi vòng lặp
$F(S)$	$ S $	Hàm mục tiêu, bằng số lớp–môn được xếp thành công trong lịch
Δ	$F(Neighbor) - F(Current)$	Mức thay đổi chất lượng giữa nghiệm lân cận và nghiệm hiện tại
P_{accept}	$\exp(\Delta / Temp)$	Xác suất chấp nhận nghiệm không cải thiện khi $\Delta \leq 0$
fast_insertion	Numba JIT	Thủ tục kiểm tra xung đột lớp, giáo viên và chèn nhanh lớp–môn vào lịch

Với hàm mục tiêu $F(S) = |S|$, ALNS ưu tiên tối đa hóa số lớp-môn được phân công hợp lệ. Tham số *destroy_ratio* quyết định mức độ phá vỡ nghiệm: giá trị nhỏ làm thuật toán loại bỏ nhiều thành phần hơn, tăng khả năng thoát khỏi tối ưu cục bộ nhưng cũng làm tăng chi phí tái xây dựng; ngược lại, giá trị lớn giúp thuật toán tìm kiếm cục bộ hơn và ổn định hơn. Trong cài đặt thực nghiệm, *destroy_ratio* = 3 tạo ra mức phá hủy tương đối mạnh, phù hợp với mục tiêu tái sắp xếp các lớp-môn khó sau khi lịch đã bị bão hòa.



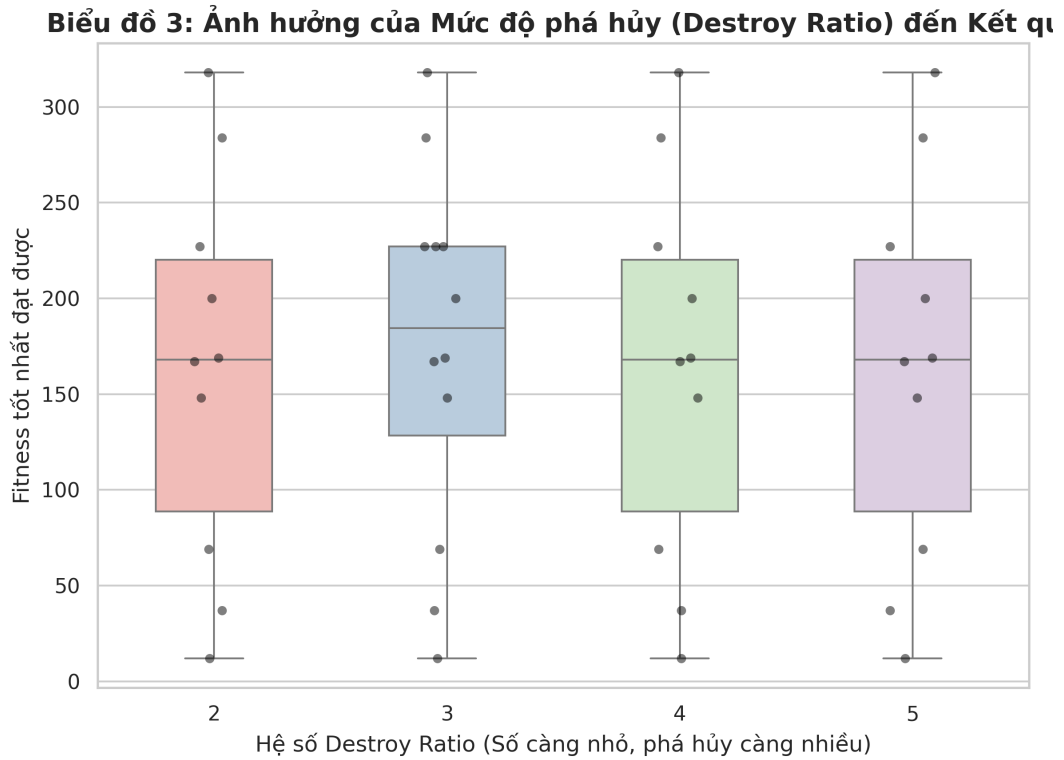
Hình 15: Đường hội tụ fitness của thuật toán ALNS trong quá trình tìm kiếm

Hình 15 cho thấy giá trị fitness tăng nhanh ở giai đoạn đầu, khi thuật toán còn nhiều vị trí trống để chen thêm lớp-môn. Sau đó đường cong có xu hướng chậm dần và xuất hiện các đoạn nằm ngang, phản ánh việc thuật toán bắt đầu gặp các xung đột tài nguyên về lớp học, giáo viên và kíp thời gian. Cơ chế chấp nhận nghiệm kém hơn theo xác suất $\exp(\Delta/Temp)$ giúp ALNS vẫn có khả năng di chuyển ra khỏi các cấu trúc lịch cục bộ, từ đó tìm được các phương án tái chen tốt hơn trong những vòng lặp sau.



Hình 16: Quan hệ giữa kích thước dữ liệu và thời gian chạy của thuật toán ALNS

Hình 16 minh họa xu hướng thời gian chạy tăng theo kích thước bài toán. Khi số lớp, số giáo viên và số môn học tăng, số lớp–môn cần xét và số khả năng chèn hợp lệ đều tăng, làm chi phí kiểm tra xung đột trong bước tái xây dựng lớn hơn. Việc sử dụng mảng băm/rảnh cho lớp và giáo viên, kết hợp với `fast_insertion` được biên dịch bằng Numba, giúp giảm đáng kể chi phí kiểm tra so với duyệt thuần Python. Tuy nhiên, do mỗi vòng lặp ALNS đều sao chép trạng thái và thử chèn lại nhiều lớp–môn chưa được xếp, thời gian vẫn tăng rõ rệt ở các bộ dữ liệu lớn.



Hình 17: Ảnh hưởng của tham số *destroy_ratio* trên bộ dữ liệu *hustack*

Hình 17 so sánh ảnh hưởng của các mức *destroy_ratio* trên bộ dữ liệu *hustack*. Kết quả cho thấy tham số này quyết định trực tiếp sự cân bằng giữa khai thác và khám phá. Khi phá hủy quá ít, thuật toán chỉ điều chỉnh nhẹ quanh nghiệm hiện tại nên khó cải thiện các lớp-môn bị kẹt. Khi phá hủy quá nhiều, nghiệm lân cận khác biệt lớn hơn nhưng chi phí tái xây dựng tăng và chất lượng có thể dao động mạnh. Do đó, lựa chọn *destroy_ratio* trung gian giúp ALNS giữ được cấu trúc tốt của lịch hiện tại, đồng thời vẫn tạo đủ khoảng trống để tái chen các lớp-môn khó.

4.5 Kết quả thực nghiệm thuật toán di truyền kết hợp bộ giải mã tham lam ngẫu nhiên

4.5.1 Thiết lập thí nghiệm

Các thí nghiệm được thực hiện trên bộ dữ liệu như đã trình bày ở 3.1. Để đánh giá ảnh hưởng của các thành phần trong thuật toán, nhóm thực hiện 8 cấu hình GA. Các cấu hình này được chia thành hai nhóm: nhóm khởi tạo theo heuristic và nhóm khởi tạo ngẫu nhiên.

Trong nhóm khởi tạo heuristic, nghiệm greedy baseline được đưa vào quần thể ban đầu, kết hợp với các cá thể sinh theo cơ chế noisy, hard-first và random. Trong nhóm khởi tạo random, quần thể ban đầu không chứa nghiệm greedy, nhằm quan sát rõ hơn khả năng tự hội tụ của thuật toán di truyền.

Các siêu tham số chung được giữ cố định như sau:

$$\text{population size} = 40, \quad \text{elite} = 5, \quad p_c = 0.85, \quad k_{\text{tournament}} = 3, \quad \text{seed} = 42.$$

Các cấu hình khác nhau chủ yếu ở chế độ khởi tạo, kiểu lai ghép và xác suất đột biến. Cụ thể, nhóm so sánh hai kiểu lai ghép OX và PMX, đồng thời khảo sát ba mức đột biến 0.40, 0.75 và 0.90.

Bảng 11: Các cấu hình thực nghiệm của thuật toán GA

STT	Cấu hình	Init	Crossover	Mutation rate	Mutation type
1	Heuristic OX + Mixed	Heuristic	OX	0.75	Mixed
2	Heuristic Mut=0.40	Heuristic	OX	0.40	Mixed
3	Heuristic Mut=0.90	Heuristic	OX	0.90	Mixed
4	Heuristic PMX + Mixed	Heuristic	PMX	0.75	Mixed
5	Random OX + Mixed	Random	OX	0.75	Mixed
6	Random Mut=0.40	Random	OX	0.40	Mixed
7	Random Mut=0.90	Random	OX	0.90	Mixed
8	Random PMX + Mixed	Random	PMX	0.75	Mixed

Các chỉ số đánh giá được sử dụng gồm:

- **Obj**: tổng số lớp-môn được xếp lịch thành công;
- **Success rate**: tỉ lệ lớp-môn được xếp lịch thành công trên tổng số lớp-môn;
- **Greedy**: kết quả của nghiệm tham lam cơ sở;
- **Gain**: độ chênh lệch giữa GA và Greedy;
- **Better/Equal/Worse**: số bộ dữ liệu mà GA tốt hơn, bằng hoặc kém Greedy;
- **AvgTime**: thời gian chạy trung bình trên một file dữ liệu.

4.5.2 Kết quả tổng hợp theo cấu hình

Bảng 12 trình bày kết quả tổng hợp của 8 cấu hình GA trên toàn bộ 60 bộ dữ liệu. Tổng số lớp-môn trong toàn bộ tập thực nghiệm là 241399. Kết quả greedy baseline xếp được 227161 lớp-môn, tương ứng với tỉ lệ thành công 94.1019%.

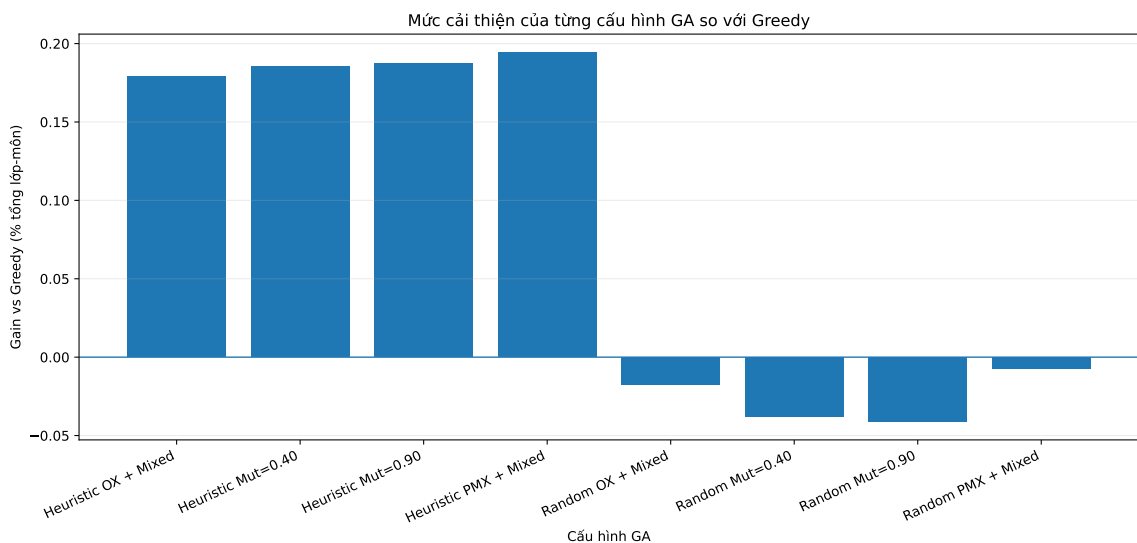
Bảng 12: Tổng hợp kết quả của các cấu hình GA trên 60 bộ dữ liệu

Cấu hình	Obj	Success(%)	Greedy(%)	Gain	Gain(%)	Better	Equal	Worse
Heuristic OX + Mixed	227593	94.2808	94.1019	432	0.1790	25	35	0
Heuristic Mut=0.40	227609	94.2875	94.1019	448	0.1856	26	34	0
Heuristic Mut=0.90	227613	94.2891	94.1019	452	0.1872	26	34	0
Heuristic PMX + Mixed	227630	94.2962	94.1019	469	0.1943	24	36	0
Random OX + Mixed	227119	94.0845	94.1019	-42	-0.0174	24	11	25
Random Mut=0.40	227070	94.0642	94.1019	-91	-0.0377	26	9	25
Random Mut=0.90	227062	94.0609	94.1019	-99	-0.0410	26	9	25
Random PMX + Mixed	227144	94.0948	94.1019	-17	-0.0070	24	11	25

Từ Bảng 12, có thể thấy nhóm cấu hình khởi tạo heuristic cho kết quả ổn định hơn rõ rệt so với nhóm khởi tạo random. Cả 4 cấu hình heuristic đều không có trường hợp nào kém hơn greedy baseline. Điều này cho thấy việc đưa nghiệm greedy vào quần thể ban đầu giúp thuật toán có một điểm xuất phát tốt, đồng thời vẫn cho phép GA cải thiện nghiệm thông qua lai ghép và đột biến.

Trong số các cấu hình heuristic, cấu hình **Heuristic PMX + Mixed** đạt kết quả tốt nhất, với 227630 lớp-môn được xếp thành công, cao hơn greedy 469 lớp-môn. Tỷ lệ thành công tổng thể đạt 94.2962%, tăng 0.1943 điểm phần trăm so với greedy. Mặc dù mức tăng tuyệt đối không quá lớn, cần lưu ý rằng greedy baseline trong bài toán này đã là một nghiệm nền rất mạnh. Do đó, việc GA tiếp tục cải thiện được trên nghiệm greedy ở quy mô 60 bộ dữ liệu cho thấy tác dụng của quá trình tiến hóa trên không gian thứ tự xếp lịch.

Ngược lại, nhóm khởi tạo random có kết quả tổng thể thấp hơn greedy. Nguyên nhân là khi không có nghiệm heuristic làm điểm neo ban đầu, GA phải tự khám phá không gian hoán vị rất lớn trong giới hạn thời gian cố định. Với các bộ dữ liệu lớn và khó, đặc biệt là *Adversarial* và *Exponential*, quần thể random dễ hội tụ chậm hơn hoặc không kịp đạt tới chất lượng của greedy baseline.



Hình 18: Mức cải thiện của từng cấu hình GA so với Greedy baseline

Hình 18 minh họa trực quan mức cải thiện của từng cấu hình. Các cấu hình heuristic đều nằm phía trên mốc 0, trong khi các cấu hình random có xu hướng dao động quanh hoặc thấp hơn greedy. Kết quả này củng cố nhận xét rằng tri thức heuristic đóng vai trò quan trọng trong việc định hướng tìm kiếm ban đầu của GA.

4.5.3 So sánh kết quả theo từng bộ dữ liệu

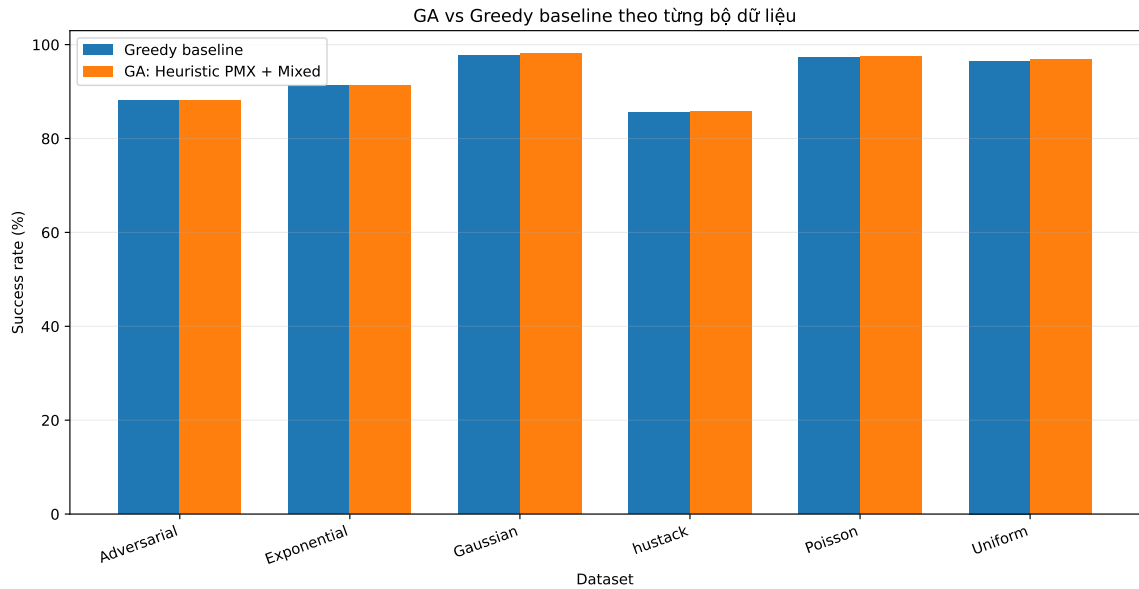
Để phân tích sâu hơn, Bảng 13 trình bày kết quả của cấu hình tốt nhất, **Heuristic PMX + Mixed**, trên từng nhóm dữ liệu.

Bảng 13: Kết quả của cấu hình Heuristic PMX + Mixed theo từng bộ dữ liệu

Dataset	Files	Obj	Total	Success(%)	Greedy(%)	Gain
Adversarial	10	42180	47899	88.0603	88.0603	0
Exponential	10	43748	47900	91.3319	91.3090	11
Gaussian	10	46976	47899	98.0730	97.7557	152
Poisson	10	46725	47899	97.5490	97.2150	160
Uniform	10	46370	47899	96.8079	96.5093	143
hustack	10	1631	1903	85.7068	85.5491	3

Kết quả cho thấy thuật toán GA cải thiện rõ nhất trên các bộ dữ liệu *Gaussian*, *Poisson* và *Uniform*. Đây là các nhóm dữ liệu mà ràng buộc phân công giáo viên không quá cực đoan, do đó việc thay đổi thứ tự xếp lịch có thể tạo ra thêm các khoảng trống hợp lệ cho những lớp-môn khó. Cụ thể, cấu hình Heuristic PMX + Mixed cải thiện 152 lớp-môn trên bộ *Gaussian*, 160 lớp-môn trên bộ *Poisson* và 143 lớp-môn trên bộ *Uniform*.

Đối với bộ *Adversarial*, GA không cải thiện so với greedy. Điều này là hợp lý vì bộ dữ liệu này được thiết kế để tạo ra tình huống khan hiếm giáo viên cục bộ, trong đó một số môn học chỉ có rất ít giáo viên có thể đảm nhiệm. Khi tài nguyên giáo viên trở thành nút thắt chính, việc thay đổi thứ tự lớp-môn bằng GA có thể không đủ để tạo ra cải thiện đáng kể nếu các ràng buộc đã gần như bão hòa.



Hình 19: So sánh tỉ lệ xếp lịch thành công giữa GA và Greedy theo từng bộ dữ liệu

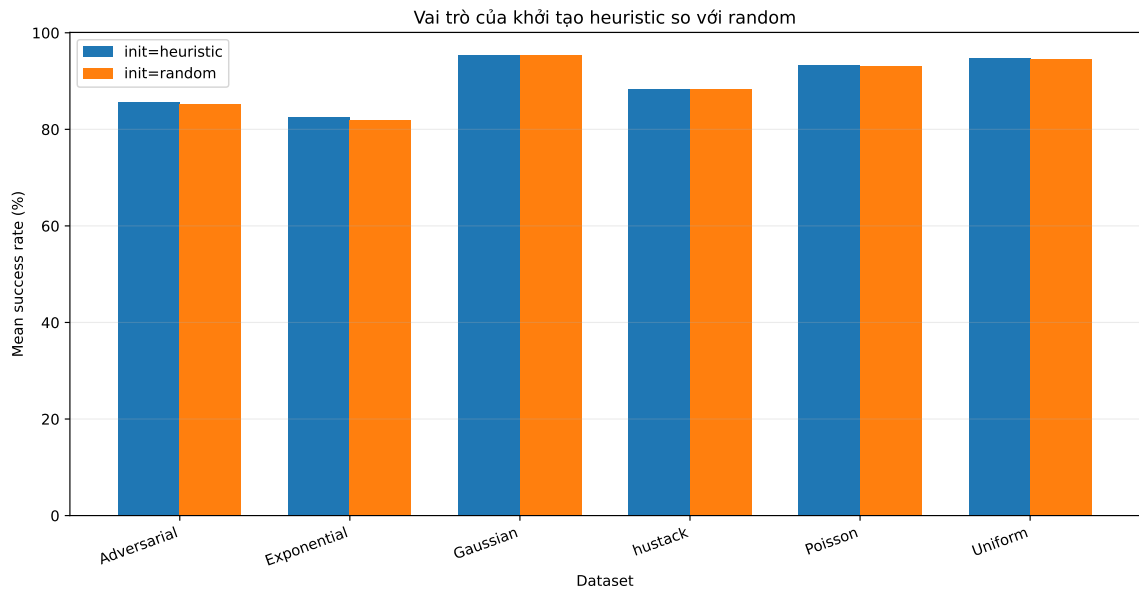
4.5.4 Ảnh hưởng của chế độ khởi tạo

Một trong những mục tiêu quan trọng của thí nghiệm là đánh giá vai trò của khởi tạo heuristic. Bảng 14 so sánh trực tiếp hai cấu hình cơ sở: **Heuristic OX + Mixed** và **Random OX + Mixed**.

Bảng 14: So sánh khởi tạo heuristic và khởi tạo random

Cấu hình	Obj	Success(%)	Gain	Better	Equal	Worse
Heuristic OX + Mixed	227593	94.2808	432	25	35	0
Random OX + Mixed	227119	94.0845	-42	24	11	25

Có thể thấy, khi sử dụng cùng toán tử OX và mixed mutation, cấu hình heuristic vượt greedy 432 lớp-môn và không có bộ dữ liệu nào kém hơn greedy. Trong khi đó, cấu hình random thấp hơn greedy 42 lớp-môn trên tổng thể và có 25/60 bộ dữ liệu cho kết quả kém hơn greedy. Kết quả này cho thấy nghiệm greedy không chỉ là một baseline để so sánh, mà còn là một thành phần hữu ích để khởi tạo quần thể. Việc đưa nghiệm tốt ban đầu vào GA giúp thuật toán tránh mất nhiều thời gian ở các vùng nghiệm chất lượng thấp.



Hình 20: So sánh ảnh hưởng của khởi tạo heuristic và random theo từng dataset

4.5.5 Ảnh hưởng của xác suất đột biến

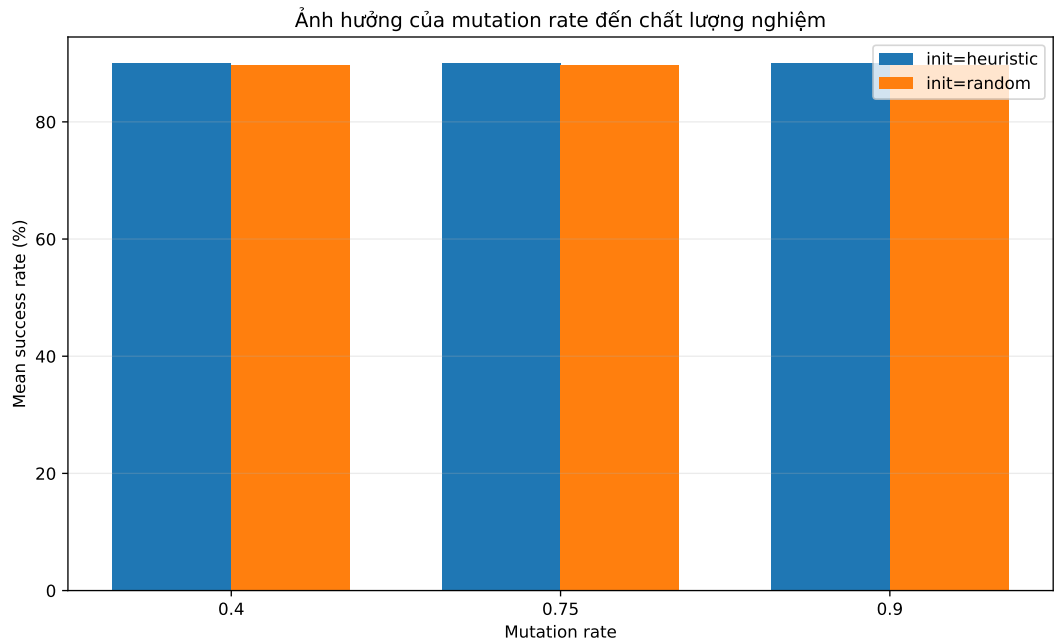
Trong bài toán này, mutation tác động trực tiếp lên thứ tự các cặp lớp-môn trong chromosome. Vì vậy, mutation có vai trò tạo đa dạng quần thể và giúp thuật toán thoát khỏi các thứ tự xếp lịch kém. Nhóm khảo sát ba mức mutation rate là 0.40, 0.75 và 0.90.

Bảng 15: So sánh các mức mutation rate trong nhóm khởi tạo heuristic

Cấu hình	Mutation rate	Obj	Success(%)	Gain	Better/Equal/Worse
Heuristic Mut=0.40	0.40	227609	94.2875	448	26/34/0
Heuristic OX + Mixed	0.75	227593	94.2808	432	25/35/0
Heuristic Mut=0.90	0.90	227613	94.2891	452	26/34/0

Ba mức mutation rate cho kết quả khá gần nhau. Mức 0.90 đạt kết quả cao nhất trong nhóm OX heuristic, với 227613 lớp-môn được xếp thành công, cao hơn greedy 452 lớp-môn. Tuy nhiên, chênh lệch giữa các mức mutation không lớn. Điều này cho thấy bộ giải mã tham lam ngẫu nhiên hóa đã đóng góp một phần đáng kể vào tính đa dạng của nghiệm, do đó thuật toán không quá nhạy với thay đổi nhỏ của mutation rate trong khoảng được khảo sát.

Về mặt giải thích thuật toán, mutation rate cao không gây suy giảm mạnh vì nghiệm tốt được bảo toàn bằng elitism. Các cá thể tốt nhất được giữ lại trực tiếp sang thế hệ sau, trong khi phần còn lại của quần thể có thể được đột biến mạnh để khám phá thêm các thứ tự xếp lịch mới.



Hình 21: Ảnh hưởng của mutation rate đến tỉ lệ xếp lịch thành công

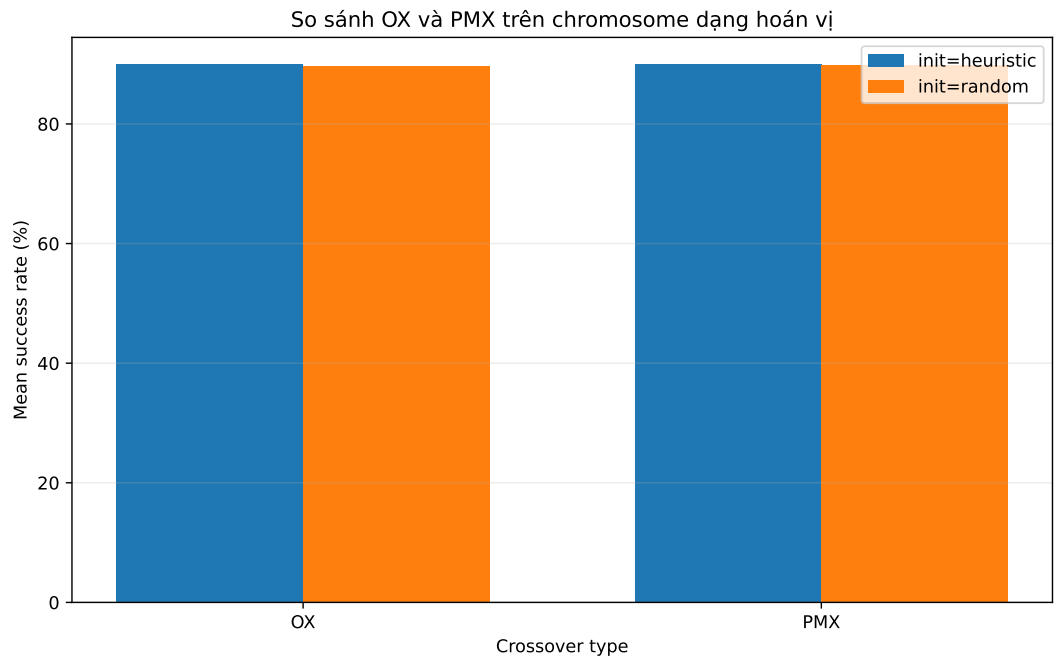
4.5.6 So sánh toán tử lai ghép OX và PMX

Do chromosome là một hoán vị của các lớp-môn, toán tử crossover cần đảm bảo cá thể con không bị mất hoặc lặp gene. Hai toán tử được so sánh là OX và PMX. Bảng 16 trình bày kết quả so sánh trong nhóm khởi tạo heuristic với mutation rate bằng 0.75.

Bảng 16: So sánh OX và PMX trong nhóm khởi tạo heuristic

Cấu hình	Crossover	Obj	Success(%)	Gain
Heuristic OX + Mixed	OX	227593	94.2808	432
Heuristic PMX + Mixed	PMX	227630	94.2962	469

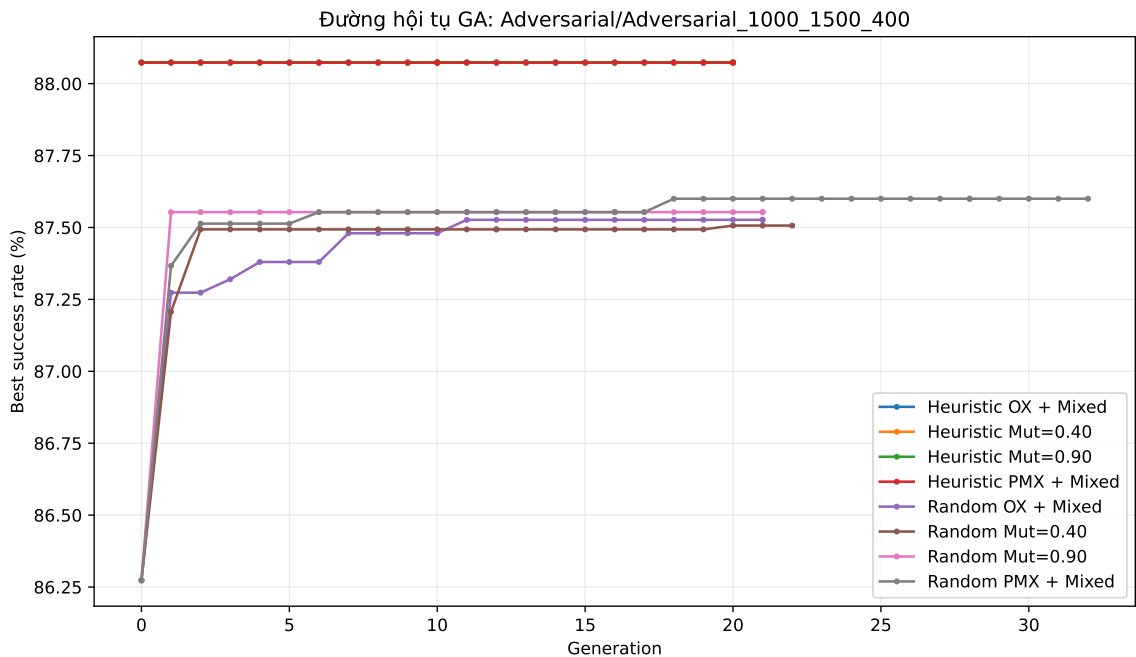
PMX cho kết quả tốt hơn OX trên tổng thể, cải thiện 469 lớp-môn so với greedy, trong khi OX cải thiện 432 lớp-môn. Sự khác biệt này không quá lớn nhưng cho thấy PMX phù hợp với bài toán trong thiết lập thí nghiệm hiện tại. Một lý do có thể là PMX giữ lại một phần ảnh xạ vị trí giữa hai cá thể cha mẹ, từ đó bảo toàn tốt hơn các quan hệ vị trí cục bộ trong hoán vị. Với bài toán xếp lịch, quan hệ thứ tự giữa các lớp-môn khó có ảnh hưởng trực tiếp đến khả năng tìm được kíp học và giáo viên hợp lệ.



Hình 22: So sánh hai toán tử lai ghép OX và PMX

4.5.7 Phân tích hội tụ

Để đánh giá rõ hơn hành vi của thuật toán GA trong quá trình tiến hóa, nhóm tiến hành vẽ đường hội tụ của cả 8 cấu hình thực nghiệm trên một số bộ dữ liệu đại diện thuộc nhóm *Adversarial*. Trục hoành biểu diễn số thế hệ, còn trục tung biểu diễn tỉ lệ lớp-môn được xếp thành công tốt nhất tính đến thế hệ hiện tại. Do thuật toán sử dụng cơ chế elitism, đường biểu diễn best fitness có dạng không giảm: nghiệm tốt nhất sau mỗi thế hệ luôn được giữ lại và không bị mất đi bởi các toán tử lai ghép hoặc đột biến.

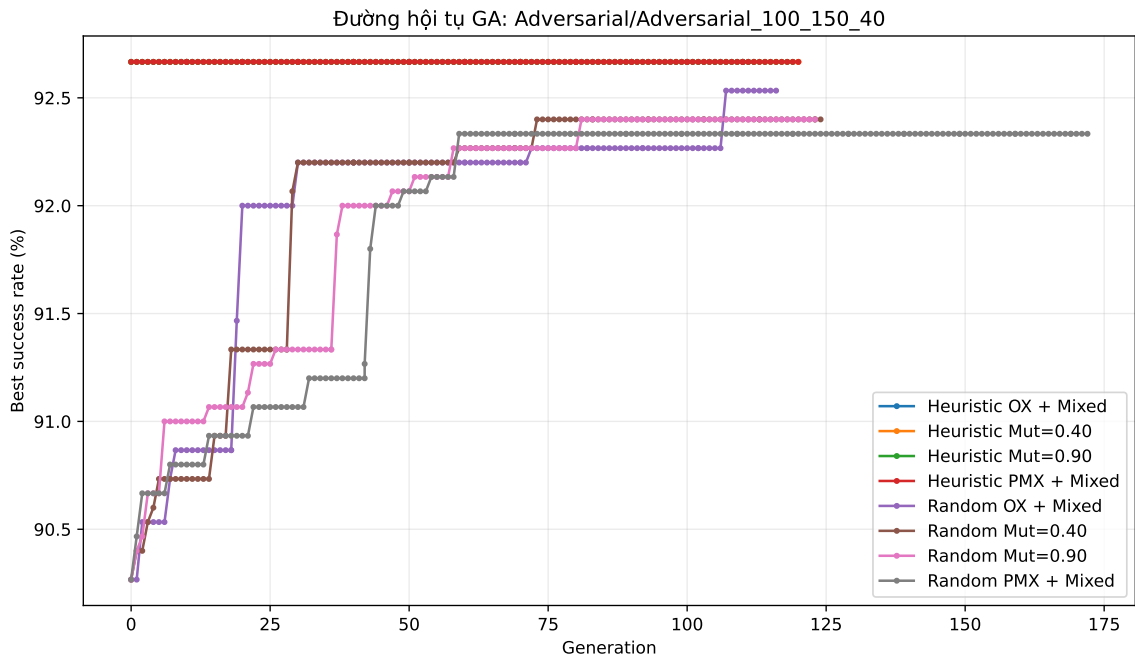


Hình 23: Đường hội tụ của 8 cấu hình GA trên bộ dữ liệu Adversarial_1000_1500_400

Hình 23 mô tả quá trình hội tụ trên bộ dữ liệu có kích thước lớn $N = 1000$, $T = 1500$, $M = 400$. Có thể thấy các cấu hình khởi tạo heuristic gần như nằm ngang ngay từ thế hệ đầu tiên. Điều này xảy ra vì trong chế độ heuristic, nghiệm greedy baseline đã được đưa trực tiếp vào quần thể ban đầu. Do đó, thuật toán không phải bắt đầu từ nghiệm ngẫu nhiên chất lượng thấp, mà xuất phát từ một nghiệm nền đã rất mạnh.

Trong hình này, cấu hình **Heuristic PMX + Mixed** đạt tỉ lệ thành công cao nhất, khoảng 88.07%, và gần như không cần thêm nhiều thế hệ để cải thiện. Các cấu hình heuristic còn lại cũng hội tụ rất nhanh và gần như trùng nhau. Điều này cho thấy trên bộ dữ liệu lớn thuộc nhóm *Adversarial*, không gian cải thiện so với greedy là rất nhỏ. Nguyên nhân là bộ *Adversarial* được thiết kế với các nút thất cục bộ về giáo viên, trong đó một số môn học có rất ít giáo viên có thể giảng dạy. Khi tài nguyên giáo viên đã gần bão hòa, việc thay đổi thứ tự lớp–môn chỉ có thể tạo ra cải thiện hạn chế.

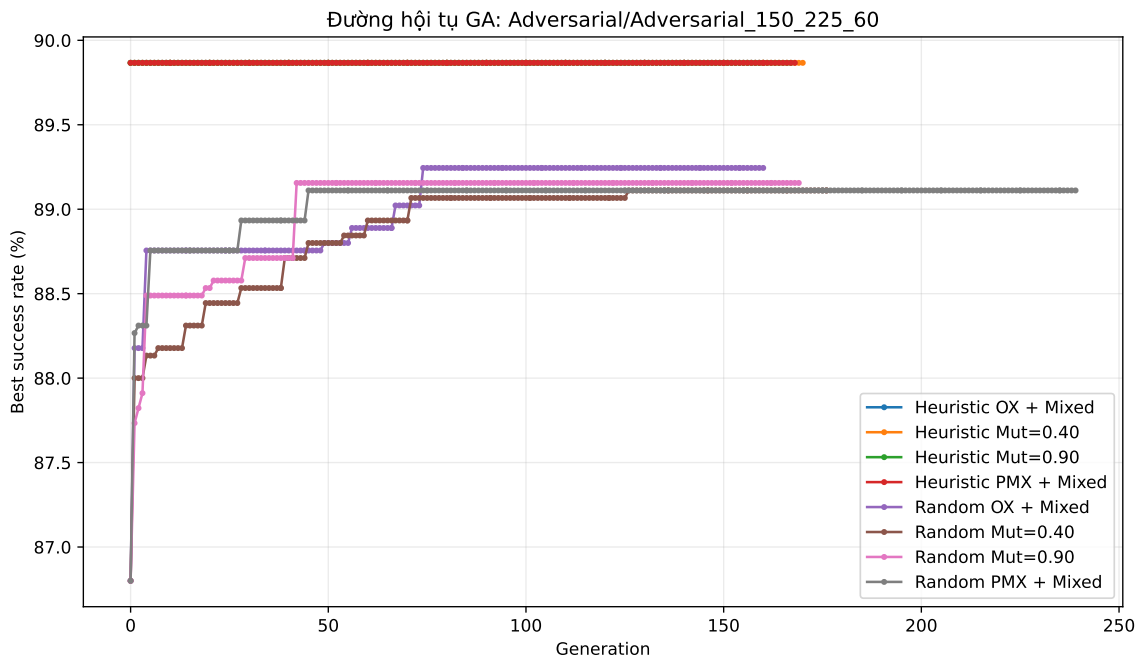
Ngược lại, nhóm khởi tạo random bắt đầu ở mức thấp hơn, khoảng 86.27%, sau đó tăng nhanh trong một vài thế hệ đầu và dần ổn định quanh mức 87.5%–87.6%. Tuy vậy, các cấu hình random vẫn không bắt kịp nhóm heuristic. Kết quả này cho thấy với dữ liệu lớn, nếu chỉ khởi tạo ngẫu nhiên thì GA cần nhiều thời gian hơn để tìm được vùng nghiệm tốt. Trong giới hạn thời gian thực nghiệm, tri thức heuristic ban đầu đóng vai trò rất quan trọng để đưa quần thể đến gần nghiệm tốt ngay từ đầu.



Hình 24: Đường hội tụ của 8 cấu hình GA trên bộ dữ liệu Adversarial_100_150_40

Hình 24 cho thấy một hành vi khác trên bộ dữ liệu nhỏ hơn với $N = 100$, $T = 150$, $M = 40$. Trong trường hợp này, các cấu hình random có nhiều cơ hội cải thiện hơn qua các thế hệ. Ban đầu, nhóm random chỉ đạt khoảng 90.3%, nhưng sau quá trình tiến hóa, một số cấu hình đã tăng lên trên 92.4%. Cụ thể, cấu hình **Random OX + Mixed** có những bước nhảy rõ rệt và đạt mức xấp xỉ 92.5% sau khoảng hơn 100 thế hệ.

Hiện tượng này cho thấy trên các bộ dữ liệu quy mô vừa và nhỏ, GA có đủ thời gian để khai thác các toán tử tiến hóa. Các phép lai ghép và đột biến có thể tạo ra các thứ tự lớp-môn tốt hơn đáng kể so với quần thể ban đầu. Đặc biệt, các đường hội tụ của nhóm random có dạng bậc thang rõ rệt: fitness giữ nguyên trong một số thế hệ, sau đó tăng đột ngột khi thuật toán tìm được một chromosome có thứ tự xếp lịch tốt hơn. Đây là đặc trưng thường gặp của GA trên bài toán hoán vị, vì một thay đổi nhỏ trong thứ tự gene có thể làm bộ giải mã tham lam tìm được thêm một số vị trí hợp lệ cho các lớp-môn khó. Tuy nhiên, cấu hình **Heuristic PMX + Mixed** vẫn đạt kết quả cao nhất ngay từ đầu, khoảng 92.67%, và giữ nguyên trong suốt quá trình chạy. Điều này cho thấy nghiệm heuristic ban đầu trong trường hợp này đã rất tốt, đến mức các toán tử tiến hóa không tìm được cải thiện thêm trong giới hạn thời gian. Nói cách khác, với bộ dữ liệu này, GA khởi tạo random thể hiện rõ khả năng học và cải thiện, nhưng GA khởi tạo heuristic vẫn có lợi thế về chất lượng nghiệm cuối cùng.



Hình 25: Đường hội tụ của 8 cấu hình GA trên bộ dữ liệu Adversarial_150_225_60

Hình 25 tiếp tục củng cố các nhận xét trên với bộ dữ liệu $N = 150$, $T = 225$, $M = 60$. Cấu hình **Heuristic PMX + Mixed** tiếp tục là cấu hình tốt nhất, đạt khoảng 89.87% ngay từ đầu và gần như không thay đổi trong suốt quá trình tiến hóa. Các cấu hình heuristic còn lại nằm rất sát nhau, trong đó **Heuristic OX + Mixed** và **Heuristic Mut=0.90** gần như trùng với nhau.

Đối với nhóm random, các đường hội tụ tăng mạnh ở giai đoạn đầu, sau đó cải thiện chậm dần. Cấu hình **Random OX + Mixed** tăng từ khoảng 86.8% lên hơn 89.2%, trong khi **Random PMX + Mixed** tăng nhanh ở các thế hệ đầu rồi ổn định quanh mức 89.1%. Điều này cho thấy các toán tử tiến hóa thực sự có tác dụng cải thiện nghiệm khi quần thể ban đầu chưa tốt. Tuy nhiên, sau một số thế hệ, các cấu hình random cũng bắt đầu bị chững lại do quần thể đã hội tụ về các thứ tự xếp lịch tương đối giống nhau.

Từ ba biểu đồ hội tụ trên, có thể rút ra một số kết luận quan trọng:

- **Khởi tạo heuristic giúp hội tụ rất nhanh.** Các cấu hình heuristic thường đạt chất lượng nghiệm cao ngay tại thế hệ đầu tiên vì quần thể đã chứa nghiệm greedy baseline. Điều này đặc biệt quan trọng với các bộ dữ liệu lớn, nơi không gian hoán vị rất lớn và thời gian tìm kiếm bị giới hạn.
- **Khởi tạo random thể hiện rõ quá trình tiến hóa hơn.** Các cấu hình random thường bắt đầu từ nghiệm thấp hơn, sau đó tăng dần theo dạng bậc thang. Điều này chứng minh rằng lai ghép và đột biến có khả năng cải thiện thứ tự xếp lịch, nhưng cần nhiều thế hệ hơn để đạt tới vùng nghiệm tốt.
- **PMX phù hợp hơn trong các thí nghiệm đại diện.** Ở cả ba biểu đồ, cấu hình **Heuristic PMX + Mixed** đều là cấu hình tốt nhất hoặc nằm trong nhóm tốt nhất. Điều này phù hợp với kết quả tổng hợp ở Bảng 12, trong đó PMX đạt tổng số lớp-môn xếp được cao nhất. PMX có thể bảo toàn tốt hơn các quan hệ ánh xạ vị trí giữa hai chromosome cha mẹ, qua đó giữ lại các đoạn thứ tự có lợi cho quá trình giải mã.
- **Mutation rate cao không làm nghiệm bị phá hủy.** Các cấu hình mutation rate 0.40, 0.75 và 0.90 cho kết quả khá gần nhau. Nguyên nhân là thuật toán sử dụng elitism để giữ lại các cá thể tốt nhất, nên dù đột biến mạnh, nghiệm tốt nhất vẫn không bị mất. Nhờ đó, mutation rate cao có thể tăng khả năng khám phá mà không làm giảm best fitness.
- **Trên dữ liệu Adversarial, cải thiện sau greedy bị giới hạn.** Các đường heuristic thường nằm ngang, cho thấy greedy baseline đã khai thác rất tốt cấu trúc dữ liệu. Với các ràng buộc khan hiếm giáo viên, số lớp-môn chưa xếp được không chỉ phụ thuộc vào thứ tự xếp lịch, mà còn bị giới hạn bởi bản chất thiếu tài nguyên của bài toán.

Nhìn chung, các biểu đồ hội tụ cho thấy thuật toán GA hoạt động đúng với kỳ vọng. Khi quần thể ban đầu yếu, GA có khả năng cải thiện nghiệm qua nhiều thế hệ. Khi quần thể ban đầu đã chứa nghiệm greedy mạnh, thuật toán hội tụ gần như tức thời và đóng vai trò tinh chỉnh để tìm thêm các thứ tự xếp lịch tốt hơn. Do đó, trong bài toán này, cách tiếp cận hiệu quả nhất không phải là dùng GA hoàn toàn ngẫu nhiên, mà là kết hợp greedy heuristic để tạo nghiệm nền tốt với GA để tiếp tục khai thác không gian hoán vị lớp-môn.

4.5.8 Thời gian chạy

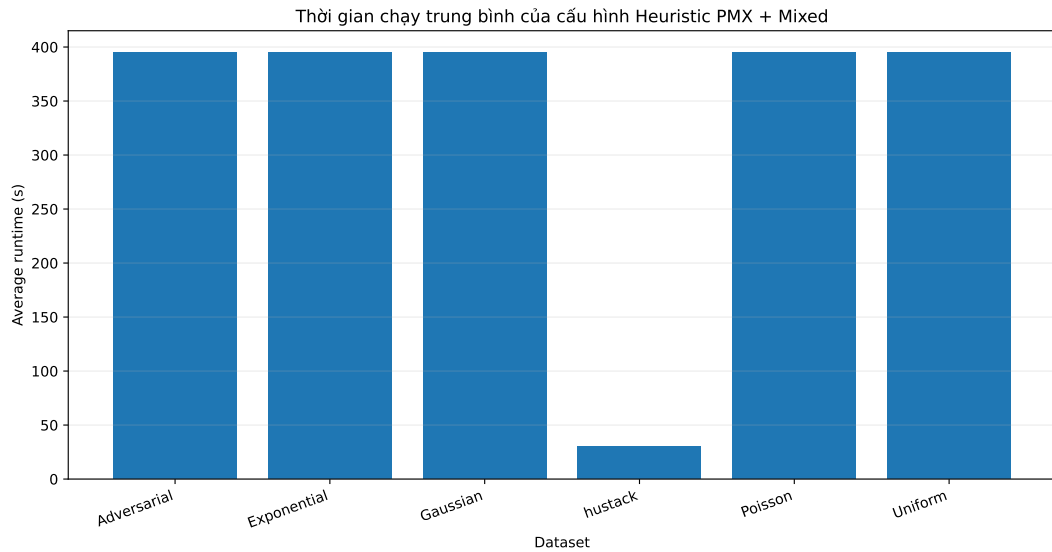
Thời gian chạy trung bình của các cấu hình GA tương đối ổn định. Các cấu hình heuristic có thời gian trung bình xấp xỉ 334.41 giây trên một file dữ liệu, trong khi các cấu hình

random xấp xỉ 334.73–334.78 giây. Riêng bộ dữ liệu *hustack* có kích thước nhỏ hơn nên thời gian chạy trung bình thấp hơn đáng kể, khoảng 30–32 giây.

Bảng 17: Thời gian chạy trung bình của một số cấu hình GA

Cấu hình	Thời gian trung bình/file (giây)
Heuristic OX + Mixed	334.42
Heuristic Mut=0.40	334.41
Heuristic Mut=0.90	334.41
Heuristic PMX + Mixed	334.41
Random OX + Mixed	334.78
Random Mut=0.40	334.76
Random Mut=0.90	334.73
Random PMX + Mixed	334.75

Do thuật toán được chạy với giới hạn thời gian theo kích thước bài toán, thời gian trung bình giữa các cấu hình gần như tương đương nhau. Vì vậy, khi so sánh các cấu hình, chất lượng nghiệm là tiêu chí quan trọng hơn thời gian chạy. Điều này cũng làm cho việc so sánh trở nên công bằng hơn, vì mỗi cấu hình có ngân sách thời gian gần như giống nhau.



Hình 26: Thời gian chạy trung bình theo từng bộ dữ liệu

4.5.9 Nhận xét chung

Từ các kết quả thực nghiệm, có thể rút ra một số nhận xét chính như sau:

- Thuật toán GA kết hợp randomized greedy decoder có thể cải thiện nghiệm greedy baseline trên tổng thể, đặc biệt khi sử dụng khởi tạo heuristic.
- Cấu hình tốt nhất là **Heuristic PMX + Mixed**, đạt 227630 lớp–môn được xếp thành công trên tổng số 241399 lớp–môn, tương ứng với tỉ lệ 94.2962%. Cấu hình này cải thiện 469 lớp–môn so với greedy baseline.

- Khởi tạo heuristic đóng vai trò rất quan trọng. Các cấu hình heuristic đều không kém greedy trên bất kỳ bộ dữ liệu nào, trong khi nhóm random có 25/60 trường hợp kém hơn greedy.
- PMX cho kết quả nhanh hơn OX trong thiết lập thí nghiệm này. Điều này cho thấy việc bảo toàn quan hệ ánh xạ vị trí trong chromosome hoán vị có thể phù hợp với bài toán xếp lịch.
- Xác suất đột biến trong khoảng 0.40–0.90 không làm thay đổi quá mạnh chất lượng nghiệm. Điều này cho thấy thuật toán tương đối ổn định với tham số mutation rate, một phần nhờ cơ chế elitism và bộ giải mã có yếu tố ngẫu nhiên.
- Trên bộ *Adversarial*, GA không cải thiện được so với greedy. Đây là bộ dữ liệu có cấu trúc ràng buộc khắc nghiệt, trong đó nút thất chính nằm ở sự khan hiếm giáo viên cho một số môn. Khi tài nguyên đã gần bão hòa, việc thay đổi thứ tự xếp lịch khó tạo ra cải thiện đáng kể.

Nhìn chung, thuật toán GA không thay thế hoàn toàn greedy, mà hoạt động hiệu quả nhất khi sử dụng greedy như một nghiệm nền tốt để khởi tạo quần thể. Khi đó, quá trình tiến hóa đóng vai trò tìm kiếm thêm các hoán vị lớp–môn tốt hơn, giúp cải thiện chất lượng lịch mà vẫn đảm bảo nghiệm sinh ra hợp lệ thông qua bộ giải mã tham lam ngẫu nhiên hóa.

5 KẾT LUẬN

Trong đề án này, nhóm đã nghiên cứu và triển khai bài toán phân công giáo viên kết hợp lập thời khóa biểu cho các lớp học. Đây là một bài toán tối ưu tổ hợp có nhiều ràng buộc thực tế, bao gồm ràng buộc về năng lực giảng dạy của giáo viên, thời lượng môn học, xung đột lịch của lớp, xung đột lịch của giáo viên và giới hạn số lớp học trong tuần. Mục tiêu chính của bài toán là tối đa hóa số lớp-môn được xếp hợp lệ trong thời khóa biểu.

Về mặt mô hình hóa, chúng em đã xây dựng được biểu diễn dữ liệu đầu vào gồm tập lớp, tập môn học, tập giáo viên, danh sách môn học của từng lớp, danh sách môn mà mỗi giáo viên có thể đảm nhiệm và thời lượng của từng môn. Từ đó, bài toán được phát biểu dưới dạng một mô hình tối ưu với các biến quyết định thể hiện việc chọn giáo viên và lớp bắt đầu cho từng cặp lớp-môn. Các ràng buộc được thiết kế nhằm đảm bảo một lớp không học hai môn cùng thời điểm, một giáo viên không dạy hai lớp cùng lúc và giáo viên được phân công phải có khả năng giảng dạy môn tương ứng.

Về mặt thực nghiệm, chúng em đã triển khai và so sánh nhiều hướng giải khác nhau. Các bộ giải chính xác như pywraplp, CP-SAT và Gurobi được sử dụng làm mốc tham chiếu cho các instance nhỏ và trung bình. Trong đó, Gurobi thể hiện khả năng chứng minh tối ưu tốt nhất trên các bài toán mà bộ giải có thể hoàn thành trong giới hạn thời gian. Tuy nhiên, khi kích thước dữ liệu tăng, các bộ giải chính xác nhanh chóng gặp khó khăn do không gian nghiệm mở rộng rất lớn.

Để xử lý các instance lớn hơn, bọn em đã xây dựng và đánh giá các thuật toán heuristic và metaheuristic, bao gồm Smart Greedy, AOS-HCLS, ALNS và thuật toán di truyền kết hợp bộ giải mã tham lam ngẫu nhiên. Thuật toán Smart Greedy cho thấy ưu điểm rõ rệt về tốc độ, có khả năng tạo nghiệm khả thi rất nhanh và là nền tảng tốt cho các thuật toán cải thiện phía sau. AOS-HCLS khai thác các toán tử Shift, Swap và Kick, đồng thời sử dụng cơ chế chọn toán tử thích nghi để điều chỉnh hướng tìm kiếm trong quá trình chạy. ALNS sử dụng cơ chế phá hủy-tái xây dựng kết hợp xác suất chấp nhận nghiệm kiểu Simulated Annealing, giúp thuật toán có khả năng thoát khỏi một số tối ưu cục bộ. Thuật toán GA khai thác biểu diễn hoán vị thứ tự các lớp-môn và sử dụng bộ giải mã tham lam ngẫu nhiên để sinh lịch hợp lệ từ mỗi cá thể.

Nhìn chung, đề án đã hoàn thành các mục tiêu chính: xây dựng mô hình bài toán, sinh và phân tích bộ dữ liệu thực nghiệm, triển khai nhiều nhóm thuật toán khác nhau, so sánh kết quả theo chất lượng nghiệm và thời gian chạy, đồng thời chỉ ra ưu nhược điểm của từng hướng tiếp cận. Kết quả thu được cho thấy các phương pháp heuristic và metaheuristic có tính thực tiễn cao, đặc biệt trong bối cảnh bài toán có quy mô lớn và các bộ giải chính xác bị giới hạn bởi thời gian tính toán.

6 Phụ lục

6.1 Mã giả thuật toán

Algorithm 1 Thuật toán AOS-HCLS Lập lịch thời khóa biểu

Require: Tập dữ liệu N, M, T, d ; Tổng số yêu cầu R_{total} ; Các siêu tham số $Iter_{max}, Nil_{max}, \rho$

Ensure: Lịch tối ưu nhất $Best_Schedule$

```

1:  $Schedule, Unassigned \leftarrow \text{SmartGreedyInitialization}()$ 
2:  $Best\_Schedule \leftarrow Schedule$ 
3:  $F_{current} \leftarrow \text{Fitness}(Schedule)$  ▷ Khởi tạo điểm số hiện tại
4:  $F_{best} \leftarrow F_{current}$  ▷ Khởi tạo Kỷ lục ban đầu
5:  $W \leftarrow \{Shift : 100.0, Swap : 100.0, Kick : 100.0\}$ 
6:  $Iter \leftarrow 0, No\_Improve \leftarrow 0$ 
7: while  $Iter < Iter_{max}$  and  $No\_Improve < Nil_{max}$  do
8:    $Iter \leftarrow Iter + 1$ 
9:    $Op \leftarrow \text{RouletteWheelSelection}(W)$ 
10:   $New\_Schedule, Changed \leftarrow \text{ApplyOperator}(Op, Schedule, Unassigned)$ 
11:   $Reward \leftarrow 0$  ▷ Mặc định phần thưởng là 0
12:  if  $Changed$  then
13:     $F_{new} \leftarrow \text{Fitness}(New\_Schedule)$ 
14:    if  $F_{new} > F_{best}$  then
15:       $Reward \leftarrow 50, No\_Improve \leftarrow 0$ 
16:       $Best\_Schedule \leftarrow New\_Schedule$ 
17:       $Schedule \leftarrow New\_Schedule, F_{best} \leftarrow F_{new}, F_{current} \leftarrow F_{new}$ 
18:    else if  $F_{new} > F_{current}$  then
19:       $Reward \leftarrow 20, No\_Improve \leftarrow No\_Improve + 1$ 
20:       $Schedule \leftarrow New\_Schedule, F_{current} \leftarrow F_{new}$ 
21:    else if  $F_{new} == F_{current}$  then
22:       $Reward \leftarrow 5, No\_Improve \leftarrow No\_Improve + 1$  ▷ Không cải thiện
23:       $Schedule \leftarrow New\_Schedule, F_{current} \leftarrow F_{new}$ 
24:    else
25:       $No\_Improve \leftarrow No\_Improve + 1$  ▷ Từ chối nghiệm xấu
26:    end if
27:  else
28:     $No\_Improve \leftarrow No\_Improve + 1$  ▷ Toán tử không tạo được lân cận hợp lệ
29:  end if
30:   $W[Op] \leftarrow \max(5.0, (1 - \rho) \cdot W[Op] + \rho \cdot Reward)$  ▷ Cập nhật trọng số AOS
31:  if  $|Best\_Schedule| == R_{total}$  then
32:    Break ▷ Đạt tối ưu tuyệt đối 100%
33:  end if
34: end while
35: return  $Best\_Schedule$ 

```

Algorithm 2 Thuật toán ALNS cho bài toán xếp thời khóa biểu

Require: Tập yêu cầu lớp-môn, danh sách giáo viên khả dụng, thời lượng môn học, các kíp bắt đầu hợp lệ

Ensure: Lịch tốt nhất $Best_Schedule$

```
1:  $Current \leftarrow \text{RandomGreedyInsertion}()$ 
2:  $Best \leftarrow Current$ 
3:  $Temp \leftarrow 100.0$ 
4: for  $iter \leftarrow 1$  to  $Iter_{max}$  do
5:    $Neighbor \leftarrow Current$ 
6:   Chọn ngẫu nhiên một tập lớp-môn đang được xếp trong  $Neighbor$ 
7:   Loại bỏ các lớp-môn này khỏi  $Neighbor$  và cập nhật lại ma trận bận/rảnh
8:    $Unassigned \leftarrow$  các lớp-môn chưa xuất hiện trong  $Neighbor$ 
9:   Xáo trộn  $Unassigned$ 
10:  for all  $(c, m) \in Unassigned$  do
11:    Xáo trộn danh sách giáo viên có thể dạy  $m$  và các kíp bắt đầu hợp lệ
12:    if tồn tại giáo viên  $t$  và kíp  $u$  không gây xung đột then
13:      Chèn  $(c, m, u, t)$  vào  $Neighbor$ 
14:    end if
15:  end for
16:   $\Delta \leftarrow F(Neighbor) - F(Current)$ 
17:  if  $\Delta > 0$  or  $\text{rand}(0, 1) < \exp(\Delta/Temp)$  then
18:     $Current \leftarrow Neighbor$ 
19:    if  $F(Current) > F(Best)$  then
20:       $Best \leftarrow Current$ 
21:    end if
22:  end if
23:   $Temp \leftarrow 0.99 \cdot Temp$ 
24: end for
25: return  $Best$ 
```

Algorithm 3 Thuật toán Smart Greedy cho bài toán phân công giáo viên và xếp thời khóa biểu

Require: Tập lớp N , tập giáo viên T , tập môn M , danh sách môn của từng lớp C_n , danh sách môn giáo viên có thể dạy A_t , thời lượng môn d_m

Ensure: Danh sách phân công $Assignments$

```

1:  $Assignments \leftarrow \emptyset$ 
2: Khởi tạo  $ClassSchedule[n][u] \leftarrow \text{false}$  với mọi lớp  $n$  và kíp  $u \in \{1, \dots, 60\}$ 
3: Khởi tạo  $TeacherSchedule[t][u] \leftarrow \text{false}$  với mọi giáo viên  $t$  và kíp  $u \in \{1, \dots, 60\}$ 
4: for all  $m \in M$  do
5:    $TeachersFor[m] \leftarrow \{t \in T \mid m \in A_t\}$ 
6:   Sắp xếp  $TeachersFor[m]$  tăng dần theo  $|A_t|$ 
7: end for
8:  $Tasks \leftarrow \{(n, m) \mid n \in N, m \in C_n\}$ 
9: Sắp xếp  $Tasks$  tăng dần theo  $|TeachersFor[m]|$ , nếu hòa thì giảm dần theo  $d_m$ 
10: for all  $(n, m) \in Tasks$  do
11:    $assigned \leftarrow \text{false}$ 
12:   for all  $t \in TeachersFor[m]$  do
13:     if  $assigned$  then
14:       break
15:     end if
16:     for  $session \leftarrow 0$  to 9 do
17:       if  $assigned$  then
18:         break
19:       end if
20:       for  $start \leftarrow 0$  to  $6 - d_m$  do
21:          $u \leftarrow 6 \cdot session + start + 1$ 
22:         if lớp  $n$  và giáo viên  $t$  đều rảnh trên các kíp  $u, \dots, u + d_m - 1$  then
23:           Đánh dấu lớp  $n$  bận trên các kíp  $u, \dots, u + d_m - 1$ 
24:           Đánh dấu giáo viên  $t$  bận trên các kíp  $u, \dots, u + d_m - 1$ 
25:            $Assignments \leftarrow Assignments \cup \{(n, m, u, t)\}$ 
26:            $assigned \leftarrow \text{true}$ 
27:           break
28:         end if
29:       end for
30:     end for
31:   end for
32: end for
33: return  $Assignments$ 

```

6.2 Bảng thống kê kết quả

Bảng 18: Bảng báo cáo kết quả thực nghiệm của các bộ giải exact solvers

Instance	Class	Teacher	Courses	pywraplp		CP-SAT		Gurobi	
				Objective	Time (ms)	Objective	Time (ms)	Objective	Time (ms)
hustack01				169	3856.5	169	41174.57	169	299.8
hustack02				167	1235.5	167	430.51	167	291.1
hustack03				37	102.3	37	57.43	37	69
hustack04				227	2241.8	227	986.47	227	478.3
hustack05				318	11135.2	318	1970.58	318	623.7
hustack06				69	1141.6	69	114.21	69	140.5
hustack07				200	3543.4	200	1421.66	200	320.6
hustack08				148	10797.4	148	3285.17	148	261
hustack09				284	9795.8	284	1000020.53	284	560.4
hustack10				12	25.4	12	13.07	12	39
Adversarial	20	30	10	150	16043.5	150	1000013.55	150	305.7
Exponential	20	30	10	170	46453.6	170	1000017.71	170	317.7
Gaussian	20	30	10	180	29458.2	180	760.19	180	259.5
Poisson	20	30	10	180	27699.8	180	2020.26	180	283.7
Uniform	20	30	10	180	5609	180	1564.81	180	281.9
Adversarial	50	75	20	479	1000326.8	535	1000038.21	545	1000038.2
Exponential	50	75	20	550	935377.1	549	1000027.82	550	1899.8
Gaussian	50	75	20	528	1000173	553	1000030.52	553	1693.1
Poisson	50	75	20	480	1000149.1	480	1000018.91	480	83082.1
Uniform	50	75	20	574	1000318.4	585	1000082.36	585	2399.4
Adversarial	80	120	32	889	1000932	948	1000096.83	948	7323.6
Exponential	80	120	32	822	1001472.9	868	1000041.67	869	6703.5
Gaussian	80	120	32	916	1002160	960	200250.26	960	6025.6

Còn tiếp ở trang sau

Bảng 18: Bảng báo cáo kết quả thực nghiệm của các bộ giải exact solvers (tiếp theo)

Instance	Class	Teacher	Courses	pywraplp		CP-SAT		Gurobi	
				Objective	Time (ms)	Objective	Time (ms)	Objective	Time (ms)
Poisson	80	120	32	847	1000540.3	909	1000047.02	910	6781.8
Uniform	80	120	32	888	1003306.8	944	1000078	944	6205.8
Adversarial	100	150	40	1073	1003778.9	1160	1000226.57	1160	19173.8
Exponential	100	150	40	1044	1000582.8	1114	1000050.88	1123	9877.9
Gaussian	100	150	40	1174	1001140.6	1200	219656.39	1200	9359.3
Poisson	100	150	40	1091	1001592.7	1160	1000066.04	1162	11961.5
Uniform	100	150	40	1145	1007428.6	1188	1000115.40	1188	11455.6
Adversarial	150	225	60	1558	1003990.4	1683	1000479.46	1739	52242.1
Exponential	150	225	60	1572	1001436.2	1644	1000093.94	1647	20960.1
Gaussian	150	225	60	1760	1002287.4	1800	453085.44	1800	21022.5
Poisson	150	225	60	1770	1003166	1795	1000149.99	1795	15036.8
Uniform	150	225	60	1725	1004877.5	1783	1000263.98	1783	26021
Adversarial	200	300	80	2191	1008888.6	1711	1000063.45	2346	49605.7
Exponential	200	300	80	1981	1003955.3	2255	1000192.80	2263	1000527.6
Gaussian	200	300	80	2359	1004815.4	2400	699533.06	2400	42450.2
Poisson	200	300	80	2337	1003441	2400	454213.40	2400	29845.6
Uniform	200	300	80	2348	1007819.4	2397	1000535.16	2397	47366.9
Adversarial	300	450	120	3216	1018336.6	673	1000344.45	3556	114738.7
Exponential	300	450	120	3190	1007019.3	3495	1000479.67	3517	643804
Gaussian	300	450	120	3579	1013570.1	3385	1001270.67	3600	104381.1

Còn tiếp ở trang sau

Bảng 18: Bảng báo cáo kết quả thực nghiệm của các bộ giải exact solvers (tiếp theo)

Instance	Class	Teacher	Courses	pywraplp		CP-SAT		Gurobi	
				Objective	Time (ms)	Objective	Time (ms)	Objective	Time (ms)
Poisson	300	450	120	3557	1007715.5	3600	818836.33	3600	55355.1
Uniform	300	450	120	3544	1010804	2893	1001452.82	3600	115661.5
Adversarial	500	750	200	–	–	–	913428.07	–	1002854.4
Exponential	500	750	200	–	–	4918	1001280.25	5890	620960.5
Gaussian	500	750	200	–	–	1817	1005353.56	6000	375595.8
Poisson	500	750	200	–	–	3459	1002895.65	6000	230071.8
Uniform	500	750	200	–	–	2540	1005351.31	6000	321566.7
Adversarial	800	1200	320	–	–	–	807886.72	–	–
Exponential	800	1200	320	–	–	3319	1004801.73	–	–
Gaussian	800	1200	320	–	–	25	1001810.11	–	–
Adversarial	1000	1500	400	–	–	–	1006615.61	–	–
Exponential	1000	1500	400	–	–	2022	1001892.20	–	–
Gaussian	1000	1500	400	–	–	–	915866.55	–	–
Poisson	1000	1500	400	–	–	56	1005224.46	–	–
Uniform	1000	1500	400	–	–	–	881309.14	–	–

Bảng 19: Tổng hợp best/average theo từng thuật toán và loại bộ dữ liệu

Dataset	Algorithm	Inst.	Best	Average	Avg. Time (s)
Adversarial	Smart Greedy	10	11681	3732.40	0.49
	ALNS	10	13353	4262.60	32.49
	AOS-HCLS	10	11390	3596.80	35.47
	GA	10	13211	4218	395.13
Exponential	Smart Greedy	10	11921	3728	0.48
	ALNS	10	14566	4431.30	28.55
	AOS-HCLS	10	11841	3674.60	51.64
	GA	10	14414	4374.80	395.12
Gaussian	Smart Greedy	10	11989	3823.50	1.37
	ALNS	10	14921	4745.30	26.14
	AOS-HCLS	10	11996	3822.10	37.54
	GA	10	14798	4697.60	395.17
Poisson	Smart Greedy	10	11979	3806.40	1.31
	ALNS	10	14805	4718	25.12
	AOS-HCLS	10	11993	3798.50	37.32
	GA	10	14619	4672.50	395.23
Uniform	Smart Greedy	10	11961	3820.90	2.40
	ALNS	10	14821	4691.90	26.64
	AOS-HCLS	10	11955	3817.10	42.27
	GA	10	14661	4637	395.32

Bảng 20: Báo cáo toàn bộ kết quả từng bài toán với bốn thuật toán heuristic

Dataset	Class	Teacher	Courses	Smart Greedy		ALNS		AOS-HCLS		GA		Gurobi
				Obj.	Time (s)	Obj.	Time (s)	Obj.	Time (s)	Obj.	Time (s)	
Adversarial	20	30	10	143	0.01	160	1.65	136	1.04	157	20.00	150
	50	75	20	533	0.04	514	6.94	485	6.10	493	90.02	545
	80	120	32	947	0.06	1050	6.37	882	9.24	1064	180.01	948
	100	150	40	1156	0.06	1395	8.36	1096	11.10	1390	180.04	1160
	150	225	60	1732	0.09	2028	12.78	1680	17.69	2022	360.03	1739
	200	300	80	2343	0.12	2706	18.12	2253	22.96	2681	360.06	2346
	300	450	120	3554	0.20	3973	27.79	3353	34.23	3924	360.07	3556
	500	750	200	5838	0.48	6684	47.54	5620	49.30	6630	600.18	–
	800	1200	320	9397	1.60	10763	83.63	9073	87.92	10608	900.64	–
	1000	1500	400	11681	2.29	13353	111.75	11390	115.10	13211	900.29	–
Exponential	20	30	10	160	0.01	190	1.42	157	0.67	188	20.00	170
	50	75	20	541	0.03	400	6.94	518	6.32	376	90.01	550
	80	120	32	855	0.05	776	10.02	833	8.73	760	180.01	869
	100	150	40	1095	0.07	1156	10.81	1007	10.39	1123	180.02	1123
	150	225	60	1625	0.11	1971	13.37	1597	16.63	1936	360.03	1647
	200	300	80	2247	0.13	2490	18.91	2162	20.88	2478	360.03	2263
	300	450	120	3509	0.19	4069	26.09	3431	28.23	4030	360.05	3517
	500	750	200	5870	0.45	7084	41.79	5820	52.11	6993	600.25	5890
	800	1200	320	9457	1.31	11611	69.36	9380	253.10	11450	900.20	–
	1000	1500	400	11921	2.45	14566	86.75	11841	119.37	14414	900.64	–
Gaussian	20	30	10	178	0.01	169	1.64	175	1.70	161	20.00	180
	50	75	20	553	0.03	688	4.08	534	6.29	687	90.01	553
	80	120	32	960	0.05	1158	5.31	946	10.73	1156	180.03	960
	100	150	40	1199	0.07	1461	6.39	1200	0.00	1449	180.02	1200
	150	225	60	1798	0.12	2219	9.19	1790	18.77	2192	360.05	1800
	200	300	80	2396	0.16	2988	13.55	2392	24.52	2935	360.03	2400
	300	450	120	3598	0.43	4469	21.68	3600	0.00	4385	360.07	3600
	500	750	200	5983	1.56	7473	38.31	5997	73.50	7405	600.18	6000
	800	1200	320	9581	4.17	11907	69.79	9591	116.92	11808	900.19	–

Còn tiếp ở trang sau

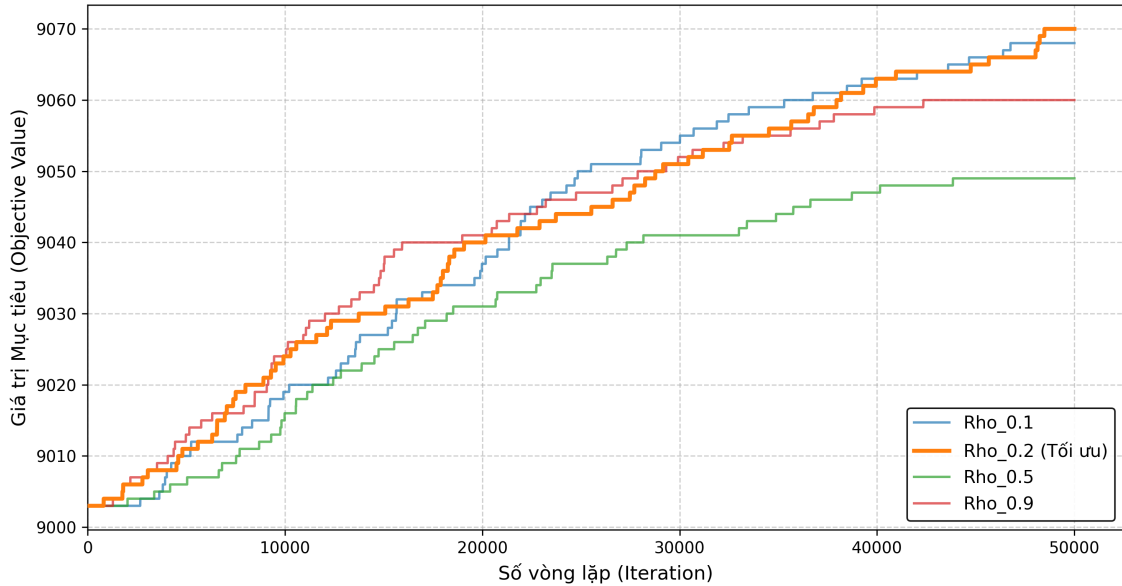
Bảng 20: Báo cáo toàn bộ kết quả từng bài toán với bốn thuật toán heuristic
(tiếp theo)

Dataset	Class	Teacher	Courses	Smart Greedy		ALNS		AOS-HCLS		GA		Gurobi
				Obj.	Time (s)	Obj.	Time (s)	Obj.	Time (s)	Obj.	Time (s)	
Poisson	1000	1500	400	11989	7.07	14921	91.46	11996	122.93	14798	901.17	–
	20	30	10	174	0.01	150	1.33	171	1.10	145	19.95	180
	50	75	20	473	0.06	620	5.52	465	5.19	620	90.01	480
	80	120	32	905	0.10	1126	5.62	869	9.03	1118	180.03	910
	100	150	40	1158	0.12	1423	6.67	1104	11.45	1417	180.04	1162
	150	225	60	1794	0.17	2212	10.58	1794	18.52	2213	360.03	1795
	200	300	80	2400	0.25	2984	13.18	2399	24.81	2958	360.09	2400
	300	450	120	3592	0.56	4497	21.40	3599	30.03	4473	360.15	3600
	500	750	200	6000	1.52	7439	37.16	5996	56.10	7329	600.42	6000
	800	1200	320	9589	4.18	11924	64.61	9595	95.86	11833	900.53	–
Uniform	1000	1500	400	11979	6.11	14805	85.10	11993	121.15	14619	901.03	–
	20	30	10	180	0.01	193	0.88	180	0.00	189	19.96	180
	50	75	20	579	0.05	661	3.93	572	6.21	648	90.01	585
	80	120	32	944	0.11	1126	6.85	940	9.69	1126	180.03	944
	100	150	40	1188	0.12	1390	7	1178	13.20	1386	180.01	1188
	150	225	60	1778	0.20	2159	11.24	1779	17.99	2137	360.03	1783
	200	300	80	2397	0.31	2954	13.72	2386	26.99	2904	360.02	2397
	300	450	120	3599	0.81	4345	22.68	3599	30.44	4277	360.17	3600
	500	750	200	5995	2.14	7397	39.01	5989	60.63	7287	600.16	6000
	800	1200	320	9588	7.71	11873	67.37	9593	136.20	11755	900.70	–
	1000	1500	400	11961	12.51	14821	93.72	11955	121.36	14661	902.08	–

Bảng 21: So sánh chi tiết ảnh hưởng của hệ số ρ trong thí nghiệm HCLS

Instance	Class	Teacher	Courses	$\rho = 0.1$		$\rho = 0.2$		$\rho = 0.5$		$\rho = 0.9$	
				Objective	Time (s)	Objective	Time (s)	Objective	Time (s)	Objective	Time (s)
Adversarial	200	300	80	2248	19.02	2248	19.16	2247	19.36	2243	19.63
Exponential	200	300	80	2169	18.15	2171	18.04	2174	18.32	2171	18.47
Gaussian	200	300	80	2397	20.70	2397	20.52	2397	20.85	2397	21.03
Poisson	200	300	80	2399	20.40	2399	20.49	2399	20.88	2399	20.90
Uniform	200	300	80	2390	20.16	2390	20.44	2391	20.52	2390	21.12
Adversarial	800	1200	320	9068	87.88	9070	91.07	9049	90.57	9060	93.02
Exponential	800	1200	320	9395	89.40	9395	89.91	9392	93.71	9393	94.80
Gaussian	800	1200	320	9589	94.89	9589	94.58	9589	97.06	9589	98.66
Poisson	800	1200	320	9596	93.65	9596	94.71	9596	97.18	9596	98.87
Uniform	800	1200	320	9592	93.00	9592	93.22	9592	96.66	9592	98.17
Adversarial	1000	1500	400	11345	115.08	11355	116.54	11348	120.75	11362	124.02
Exponential	1000	1500	400	11854	126.78	11850	129.67	11852	128.52	11854	135.52
Gaussian	1000	1500	400	11992	129.66	11992	131.63	11992	136.07	11993	140.25
Poisson	1000	1500	400	11991	124.60	11991	126.04	11991	130.46	11991	134.66
Uniform	1000	1500	400	11948	123.24	11946	125.17	11946	127.96	11945	130.86

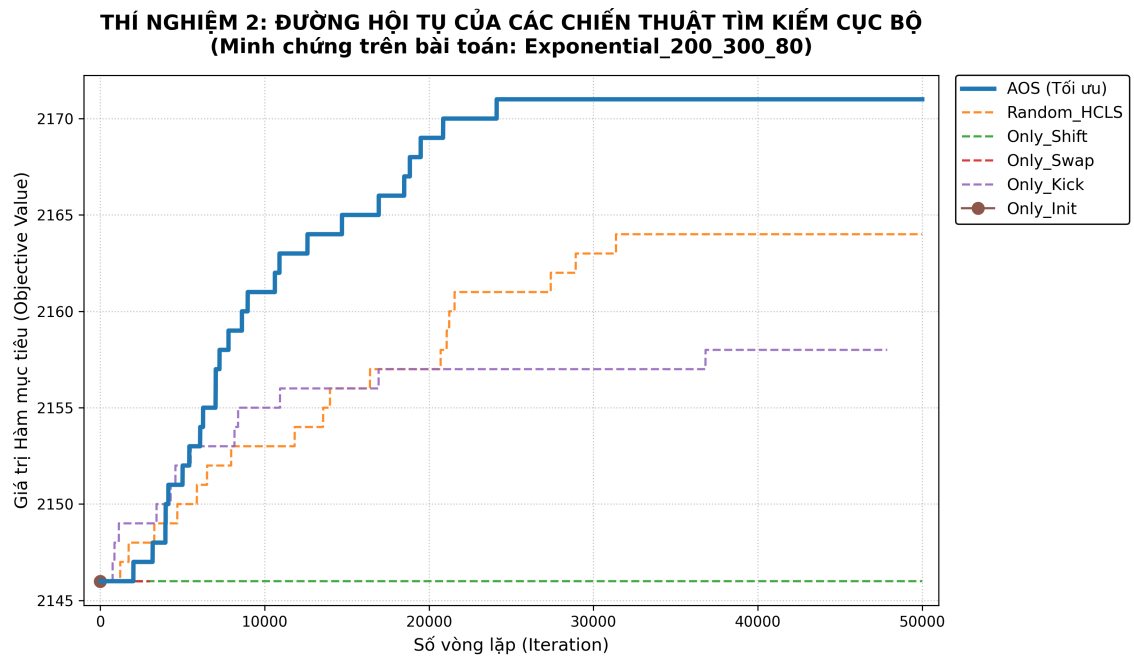
THÍ NGHIỆM 1: ĐƯỜNG HỘI TỤ CỦA CÁC HỆ SỐ HỌC TẬP (RHO)
(Minh chứng trên bài toán: Adversarial_800_1200_320)



Hình 27: Đường hội tụ của các cấu hình hệ số ρ trong thí nghiệm HCLS

Bảng 22: So sánh chi tiết các chiến lược toán tử trong thí nghiệm HCLS

Instance	Class	Teacher	Courses	AOS-HCLS		Only Init		Only Kick		Only Shift		Only Swap		Random HCLS	
				Objective	Time (s)	Objective	Time (s)	Objective	Time (s)	Objective	Time (s)	Objective	Time (s)	Objective	Time (s)
Adversarial	200	300	80	2248	29.16	2223	0.00	2236	13.51	2223	19.52	2223	3.43	2247	32.78
Exponential	200	300	80	2171	26.91	2146	0.00	2158	14.28	2146	18.49	2146	3.01	2164	27.72
Gaussian	200	300	80	2397	31.76	2397	0.00	2397	1.11	2397	21.56	2397	4.02	2397	32.59
Poisson	200	300	80	2399	35.09	2399	0.00	2399	0.95	2399	21.08	2399	3.31	2399	36.95
Uniform	200	300	80	2390	33.67	2390	0.00	2390	1.60	2390	21.25	2390	3.37	2390	31.22
Adversarial	800	1200	320	9070	156.38	9003	0.00	9053	72.26	9003	98.37	9003	16.75	9065	161.99
Exponential	800	1200	320	9395	151.03	9385	0.00	9390	56.65	9385	104.17	9385	16.12	9399	158.66
Gaussian	800	1200	320	9589	177.63	9589	0.00	9589	8.56	9589	117.17	9589	19.45	9589	207.29
Poisson	800	1200	320	9596	166.05	9596	0.00	9596	5.09	9596	107.53	9596	16.55	9596	177.97
Uniform	800	1200	320	9592	164.58	9592	0.00	9592	8.11	9592	118.42	9592	18.60	9592	184.25
Adversarial	1000	1500	400	11355	202.46	11307	0.00	11339	107.93	11307	143.92	11307	22.55	11351	229.32
Exponential	1000	1500	400	11850	221.64	11844	0.00	11853	49.84	11844	144.15	11844	22.45	11854	245.54
Gaussian	1000	1500	400	11992	227.71	11992	0.00	11992	17.24	11992	168.59	11992	24.93	11993	252.39
Poisson	1000	1500	400	11991	229.37	11991	0.00	11991	16.63	11991	161.65	11991	23.10	11991	243.91
Uniform	1000	1500	400	11946	253.94	11944	0.00	11946	34.02	11944	173.26	11944	25.84	11947	274.80



Hình 28: Đường hội tụ của các chiến lược toán tử trong thí nghiệm HCLS

Tài liệu tham khảo